MACHINE LEARNING INTERVIEW PREPARATION

LLM Fine-tuning & Alignment: Top 51 Interview Questions & Answers

Fine-Tuning, PEFT, RLHF, DPO/GRPO, Merging & Production Training

Target Persona: Senior AI Engineer / Applied AI Engineer (~3 YOE)

Focus Areas: 7B VRAM/LoRA/SFT-loss hand calculations, RLHF/DPO/GRPO trade-offs, model merging, production monitoring

Includes: Conversational + Deep-Dive Answers, Key Formulas, Common Mistakes, Follow-Ups, Final Revision Sheet

LLM Fine-tuning & Alignment – Top 51 Interview Questions & Answers

---

1. Fine-Tuning Fundamentals & Distributed Training (Q1–Q7)

Question 1: When would you choose full fine-tuning over parameter-efficient fine-tuning?

[ESSENTIAL]

Conversational Answer

"I'd reach for full fine-tuning when the task requires a genuinely large shift in the model's behavior — things like teaching it a new language, a very different domain vocabulary, or a reasoning style that's far from what the base model already does — and I have the compute budget and enough high-quality data to support it without overfitting. For most production use cases, though, I'd default to PEFT methods like LoRA first: the task is usually 'steer an already-capable model toward my domain,' not 'relearn from scratch,' and PEFT gets 90%+ of the quality at a fraction of the VRAM and storage cost. I'd only escalate to full fine-tuning if PEFT plateaus below the quality bar I need."

Intuitive Example

- Fine-tuning a base model to speak fluently in a low-resource language it barely saw in pretraining is a large distributional shift — a good candidate for full fine-tuning. Fine-tuning a model to follow your company's support-ticket formatting conventions is a small, narrow shift — a good candidate for LoRA.

Key Interview Points

- **Full Fine-Tuning**: Updates every parameter; maximum expressiveness, maximum VRAM/storage cost.
- **PEFT (LoRA/Adapters)**: Updates a small added parameter set; near-full quality on narrow shifts at a fraction of the cost.
- **Task Distributional Shift**: The larger the gap between pretraining distribution and target task, the more full fine-tuning's extra capacity matters.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Full fine-tuning trains all Ψ parameters, carrying the full 16Ψ-byte mixed-precision training memory footprint (Module 01). LoRA trains only $2rd$ parameters per adapted matrix ($r \ll d$), so its trainable-parameter and optimizer-state cost is a small fraction of Ψ, even though the frozen base weights still occupy VRAM.

Production Perspective & Trade-offs

Full fine-tuning means one full-sized checkpoint per task/customer — expensive to store and serve. PEFT adapters are megabytes, so you can keep dozens of task-specific adapters on top of one shared frozen base, swapping them per request. The trade-off is quality ceiling: for tasks requiring deep behavioral change, LoRA's low-rank update can underfit relative to full fine-tuning.

Common Mistakes

1. Assuming PEFT is always "good enough" without validating on the actual target task — narrow-rank updates can genuinely underfit large distributional shifts.
2. Defaulting to full fine-tuning "to be safe," ignoring the multi-tenant storage/serving cost of one full checkpoint per use case.

COMMON FOLLOW-UP QUESTIONS

> **Q: Can you combine full fine-tuning and LoRA in the same pipeline?**
>
> Yes — a common pattern is a full-fine-tuned (or continued-pretrained) base checkpoint, with LoRA adapters layered on top for per-customer or per-task customization.

> **Q: Does full fine-tuning risk catastrophic forgetting more than LoRA?**
>
> Yes, generally — updating every parameter can overwrite general capabilities more aggressively than a constrained low-rank update, which is one reason LoRA is often the safer default.

One-Line Takeaway

> **Takeaway:** Default to PEFT for narrow, steerable tasks; reach for full fine-tuning only when the task demands a genuinely large shift in model behavior and you have the budget to match.

Question 2: Why does the Adam optimizer state dominate fine-tuning VRAM cost?

[ESSENTIAL]

Conversational Answer

"Adam doesn't just store the model's parameters — for every single parameter, it also tracks a running first moment (momentum) and a running second moment (variance), both usually in fp32. So on top of the 2 bytes/param for fp16 weights and 2 bytes/param for fp16 gradients, you're carrying 4+4+4 = 12 bytes/param just for the fp32 master weights and Adam's two moment buffers. That's why the optimizer state, not the model weights themselves, ends up being the single biggest chunk of training memory — 12 out of the total 16 bytes per parameter in the standard mixed-precision recipe."

Intuitive Example

- For a 1B-parameter model, the raw fp16 weights are only 2GB — but Adam's fp32 master weights + momentum + variance buffers add another 12GB on top, nearly 6x the weight size alone.

Key Interview Points

- **First Moment (m)**: Exponential moving average of gradients (momentum) — 4 bytes/param in fp32.
- **Second Moment (v)**: Exponential moving average of squared gradients (adaptive LR) — 4 bytes/param in fp32.
- **FP32 Master Weights**: A full-precision copy of the parameters kept for numerically stable updates — 4 bytes/param.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Memory}_{\text{bytes}} = \underbrace{2\Psi}_{\text{fp16 params}} + \underbrace{2\Psi}_{\text{fp16 grads}} + \underbrace{4\Psi + 4\Psi + 4\Psi}_{\text{fp32 master + Adam } m,v} = 16\Psi \text{ bytes}$$

The optimizer-state portion (12Ψ) is 75% of the total 16Ψ — larger than the params and gradients combined.

Production Perspective & Trade-offs

This is exactly why ZeRO-1 (which shards only the optimizer state across GPUs) already recovers most of ZeRO's memory benefit before touching gradients or parameters — it's targeting the largest single

line item. It's also why memory-efficient optimizers (8-bit Adam, paged optimizers in QLoRA) specifically target this term.

Common Mistakes

1. Assuming the model's own parameter size ("it's a 1B model, that's 2-4GB") is a good proxy for training memory — optimizer state routinely triples or quadruples that number.
2. Forgetting that SGD with momentum has a much smaller optimizer-state footprint (just m, no v) — the 16Ψ figure is Adam/AdamW-specific.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why does ZeRO-1 alone recover so much of the memory benefit?**
>
> Because it shards the largest term (12Ψ optimizer state) across N GPUs first, before touching the smaller gradient and parameter terms.

> **Q: How does 8-bit Adam reduce this cost?**
>
> It stores the momentum/variance buffers in int8 instead of fp32, cutting the 8Ψ moment-buffer portion roughly 4x, at the cost of some quantization noise in the optimizer statistics.

One-Line Takeaway

> **Takeaway:** Adam's fp32 master weights plus momentum and variance buffers add up to 12Ψ bytes — three-quarters of the total 16Ψ training memory footprint, dwarfing the raw parameter size.

Question 3: How would you estimate the GPU memory required to full-fine-tune a 7B model?

[ESSENTIAL]

Conversational Answer

"I'd break it into two buckets: static memory and activation memory. Static memory is params + gradients + optimizer state, which for standard mixed-precision Adam training is a fixed 16Ψ bytes regardless of batch size — for a 7B model that's $16 \times 7 \times 10^9 \approx 112$ GB, before you've even run a forward pass. On top of that, activation memory scales with batch size, sequence length, and number of layers — it's the variable piece that depends on your actual training config, and it's the term gradient checkpointing specifically targets to shrink. So my estimate would start with the fixed 112GB static number, then add a batch-size-dependent activation estimate — and immediately conclude a single 80GB GPU can't hold this without sharding, mixed strategies, or memory-saving tricks."

Intuitive Example

- 112GB of static memory alone already exceeds a single H100's 80GB — before adding any activation memory — which is exactly why 7B+ full fine-tuning is always paired with ZeRO/FSDP sharding across multiple GPUs in practice.

Key Interview Points

- **Static Memory (16Ψ)**: Params + gradients + optimizer state — fixed regardless of batch size.
- **Activation Memory**: Scales with batch size × sequence length × hidden dim × layers — the variable, tunable term.
- **Gradient Checkpointing**: Trades recomputation FLOPs for activation memory savings, without touching the fixed 16Ψ term.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Memory}_{\text{static}} = 16\Psi \text{ bytes}, \qquad \Psi = 7 \times 10^9 \Rightarrow \text{Memory}_{\text{static}} = 112 \text{ GB}$$

Activation memory has no single universal formula (it depends on architecture, batch size B, sequence length L, hidden size d, and number of layers), but it grows roughly linearly with $B \times L$ for a fixed model — real profiling (measuring `torch.cuda.max_memory_allocated()` before/after a forward pass, as in this topic's notebook 01) is the reliable way to pin down the actual number for a given config, rather than a closed-form estimate.

Production Perspective & Trade-offs

This is precisely the calculation that motivates ZeRO/FSDP: sharding the static 16Ψ term across $N = 8$ GPUs takes 112GB down to 14GB per GPU (ZeRO-3), comfortably fitting even after adding activation memory. Skipping this estimate and just "trying it" on a single GPU is a common way to burn hours discovering an OOM at step one.

Common Mistakes

1. Only counting parameter memory (14GB in fp16 for a 7B model) and being surprised when training OOMs — the optimizer state alone is 6x that.
2. Treating activation memory as negligible — for long sequences or large batches it can rival or exceed the static term, especially without gradient checkpointing.

COMMON FOLLOW-UP QUESTIONS

> **Q: How would gradient checkpointing change this estimate?**
>
> It reduces the activation-memory term substantially (often by the square root of the number of layers) by discarding most intermediate activations and recomputing them during the backward pass, at the cost of roughly 20-30% more compute time.

> **Q: If you only had 2 GPUs, could you still full-fine-tune this 7B model?**
>
> With ZeRO-3 across 2 GPUs, static memory would be $112/2 = 56$GB per GPU — still likely too tight with activations on 80GB cards, so you'd probably also need gradient checkpointing or a PEFT method instead.

One-Line Takeaway

> **Takeaway:** Estimate full fine-tuning memory as a fixed 16Ψ-byte static cost (112GB for a 7B model) plus a batch/sequence-dependent activation cost — and expect to need sharding or PEFT the moment Ψ crosses a few billion.

Question 4: What's the difference between Data, Tensor, and Pipeline Parallelism?

[ESSENTIAL]

Conversational Answer

"They shard different things. Data Parallelism replicates the *entire* model on every GPU and splits the *batch* across GPUs — each GPU does a full forward/backward on its slice of data, then gradients are synchronized. Tensor Parallelism instead splits *individual weight matrices* across GPUs, so a single matrix multiply is computed cooperatively across devices — every GPU sees every token, but only owns a slice of each layer's math. Pipeline Parallelism splits the model *by layer*, assigning different contiguous blocks of layers to different GPUs, so a single forward pass flows through GPUs in sequence like an assembly line. In practice, large-scale training combines all three — data parallelism across nodes, tensor parallelism within a node for wide layers, pipeline parallelism across node groups for very deep models."

Intuitive Example

- Data Parallelism: 8 workers each build the same full car, in parallel, one car per worker per day. Tensor Parallelism: 8 workers together assemble one engine, each doing a piece of the wiring simultaneously. Pipeline Parallelism: 8 workers stand along a car assembly line, each responsible for a different section the car passes through in sequence.

Key Interview Points

- **Data Parallelism**: Same full model per GPU, different data shards — simplest, but requires the full model to fit on one GPU.
- **Tensor Parallelism**: Splits individual matrix operations across GPUs — needed when a single layer is too large for one GPU, at the cost of frequent all-reduce communication.
- **Pipeline Parallelism**: Splits layers across GPUs sequentially — reduces per-GPU parameter memory, but introduces "bubble" idle time unless micro-batched carefully.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Unlike ZeRO/FSDP (which shard the optimizer's *bookkeeping* while every GPU still runs the full forward/backward computation), Tensor and Pipeline Parallelism shard the *computation itself*. TP shrinks per-GPU activation and weight memory for the sharded layers by roughly $1/N_{\text{TP}}$; PP shrinks per-GPU parameter memory by roughly $1/N_{\text{stages}}$.

Production Perspective & Trade-offs

TP requires very fast interconnect (NVLink) since every sharded matmul needs an all-reduce — it's typically confined to GPUs within a single node. PP has lower communication overhead but wastes GPU time in the "bubble" at the start/end of each batch unless micro-batch count is tuned high enough to keep the pipeline full. DP is the easiest to reason about but does nothing to help if a single replica doesn't fit on one GPU.

Common Mistakes

1. Confusing Tensor Parallelism with ZeRO-3 — TP splits the actual matrix computation; ZeRO-3 splits which GPU *owns* (stores) each parameter shard while still computing the full operation collectively via all-gather.
2. Assuming Pipeline Parallelism scales for free — pipeline bubbles waste real GPU-time if the number of micro-batches isn't large relative to the number of pipeline stages.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why does Tensor Parallelism need fast interconnect while Pipeline Parallelism doesn't as much?**
>
> TP requires an all-reduce after every sharded matmul (many times per layer), which is latency-sensitive; PP only passes activations once between adjacent pipeline stages per micro-batch, a much lower communication frequency.

> **Q: Can you combine all three with ZeRO/FSDP simultaneously?**
>
> Yes — this is standard "3D parallelism" at frontier scale: ZeRO/FSDP data-parallel groups combined with tensor and pipeline parallelism within and across nodes.

One-Line Takeaway

> **Takeaway:** Data Parallelism replicates the model and splits data; Tensor Parallelism splits individual layer computations; Pipeline Parallelism splits the model by layer across a sequential chain of GPUs.

Question 5: How do ZeRO stages 1, 2, and 3 differ in what they shard?

[ESSENTIAL]

Conversational Answer

"ZeRO progressively shards more of the 16Ψ-byte training memory footprint across GPUs as you go from stage 1 to 3. ZeRO-1 shards only the optimizer states — the biggest single chunk (12 of the 16 bytes/param) — so it's the easiest, lowest-risk win. ZeRO-2 additionally shards the gradients. ZeRO-3 goes all the way and shards the parameters themselves, meaning no single GPU permanently holds the full model — it all-gathers the parameters it needs just-in-time for each layer's forward and backward pass, then releases them again. Each stage trades a bit more communication overhead for a bigger memory win, with ZeRO-3 giving the largest reduction but the most communication."

Intuitive Example

- Going from ZeRO-1 to ZeRO-3 on an 8-GPU 7B-model job takes per-GPU static memory from 38.5GB down to just 14GB — the difference between "tight but fits" and "comfortable" on an 80GB card.

Key Interview Points

- **ZeRO-1**: Shards optimizer states (12Ψ) — biggest win for the least communication overhead.
- **ZeRO-2**: Additionally shards gradients (2Ψ) — further reduces memory, modest extra communication.
- **ZeRO-3**: Additionally shards parameters (2Ψ) — full sharding, most communication (all-gather on every forward/backward).

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Memory}_{\text{ZeRO-1}} = 2\Psi + 2\Psi + \frac{12\Psi}{N}, \quad \text{Memory}_{\text{ZeRO-2}} = 2\Psi + \frac{14\Psi}{N}, \quad \text{Memory}_{\text{ZeRO-3}} = \frac{16}{N}$$

For a 7B model ($\Psi = 7 \times 10^9$) on $N = 8$ GPUs: ZeRO-1 $\approx$ 38.5GB/GPU, ZeRO-2 $\approx$ 26.25GB/GPU, ZeRO-3 = 14GB/GPU — versus 112GB unsharded (DDP).

Production Perspective & Trade-offs

ZeRO-3's just-in-time all-gather means every layer's forward and backward pass requires fresh communication — this is fine on fast NVLink-connected nodes but can become the bottleneck on slower interconnects, at which point ZeRO-2 (which keeps parameters fully replicated, avoiding per-layer all-gather) might actually train faster despite using more memory.

Common Mistakes

1. Assuming higher ZeRO stage is strictly "better" — it's a memory-for-communication trade, and ZeRO-3 can be slower than ZeRO-2 on bandwidth-constrained clusters despite using less memory.
2. Forgetting that ZeRO doesn't reduce total FLOPs — it only changes memory distribution and communication pattern, not the underlying compute cost of training.

COMMON FOLLOW-UP QUESTIONS

> **Q: If you have plenty of GPU memory but limited interconnect bandwidth, which ZeRO stage would you pick?**
>
> Likely ZeRO-1 or ZeRO-2 — avoiding ZeRO-3's frequent per-layer all-gather keeps communication overhead lower when bandwidth, not memory, is the bottleneck.

> **Q: Does ZeRO change the effective batch size or optimizer math?**
>
> No — ZeRO is purely a memory-placement/communication strategy; the mathematical training dynamics (gradients, optimizer updates) are identical to unsharded training.

One-Line Takeaway

> **Takeaway:** ZeRO-1 shards optimizer states, ZeRO-2 adds gradients, ZeRO-3 adds parameters — each stage trades more communication for a larger per-GPU memory reduction.

Question 6: How does FSDP differ from ZeRO-style sharding, and when would you reach for each?

[ESSENTIAL]

Conversational Answer

"Conceptually they're solving the same problem the same way — FSDP is PyTorch's native implementation, and it's very close to ZeRO-3 in behavior: it shards parameters, gradients, and

optimizer state across GPUs and all-gathers what's needed just-in-time. The practical difference is ecosystem and configurability. DeepSpeed's ZeRO gives you explicit, independently selectable stages (1, 2, or 3) plus extras like CPU/NVMe offloading. FSDP is PyTorch-native, so it integrates more seamlessly if you're already in a pure PyTorch stack without DeepSpeed as a dependency, and it configures sharding primarily through wrapping policies rather than discrete numbered stages. In practice, I'd pick FSDP for a PyTorch-native pipeline where I want fewer external dependencies, and DeepSpeed/ZeRO when I need fine-grained stage control or its offloading features."

Intuitive Example

- A team already using `torch.distributed` and Hugging Face `accelerate` with no other DeepSpeed dependencies would likely reach for FSDP directly; a team that already has a DeepSpeed config for other jobs (or needs CPU offloading for an extremely large model) would stick with ZeRO.

Key Interview Points

- **FSDP**: PyTorch-native, ZeRO-3-equivalent sharding, configured via wrapping policies.
- **DeepSpeed ZeRO**: Explicit numbered stages (1/2/3), plus CPU/NVMe offloading options.
- **Functional Equivalence**: Both shard parameters/gradients/optimizer-state and all-gather just-in-time — the choice is largely ecosystem/tooling, not a different underlying algorithm.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Both target the same 16Ψ-byte memory reduction via the same $16\Psi/N$ scaling as full ZeRO-3 sharding — there is no separate formula for FSDP; it implements the same sharding math natively in PyTorch.

Production Perspective & Trade-offs

FSDP's wrapping-policy configuration (deciding which submodules get individually sharded/wrapped) requires some tuning to get communication-efficient — wrapping too finely adds overhead, too coarsely limits memory savings. DeepSpeed's stage system is more prescriptive out of the box, which can be easier to reason about for teams new to sharded training.

Common Mistakes

1. Treating FSDP and ZeRO-3 as fundamentally different algorithms — they're different implementations of essentially the same sharding strategy.
2. Assuming FSDP lacks offloading — modern FSDP does support CPU offloading, though DeepSpeed's NVMe offloading (ZeRO-Infinity) is more mature for extreme-scale cases.

COMMON FOLLOW-UP QUESTIONS

> **Q: Does switching between FSDP and DeepSpeed change training results?**
>
> No — both implement mathematically equivalent sharded data-parallel training; results should match up to floating-point/communication-order non-determinism, not due to any algorithmic difference.

> **Q: Which would you choose for a Hugging Face Trainer-based pipeline?**
>
> Either works — Hugging Face `accelerate`/`Trainer` supports both FSDP and DeepSpeed as backends; the choice comes down to existing infra and whether you need DeepSpeed-specific features.

One-Line Takeaway

> **Takeaway:** FSDP and ZeRO-3 shard memory the same way — the real choice is ecosystem fit (PyTorch-native vs. DeepSpeed's staged, offload-capable tooling), not a difference in the underlying algorithm.

Question 7: What's the trade-off between micro-batch size and gradient accumulation steps?

[ESSENTIAL]

Conversational Answer

"Gradient accumulation lets you simulate a large effective batch size without needing the GPU memory for it all at once — you run several small 'micro-batches' forward and backward, accumulating gradients in the `.grad` buffers without calling `optimizer.step()`, and only step the optimizer after the full set of micro-batches. The trade-off is: a smaller micro-batch size keeps peak activation memory low (since only one micro-batch's activations are live at a time) but takes more sequential forward/backward passes to reach the same effective batch size, adding wall-clock time. A larger micro-batch size is faster per effective batch (fewer accumulation steps, better GPU utilization) but risks OOM if it doesn't fit in memory."

Intuitive Example

- Wanting an effective batch size of 64 on a GPU that only comfortably fits a micro-batch of 8 means running 8 accumulation steps per optimizer update — same final gradient direction as a true batch-64 forward pass, just computed in eight smaller pieces.

Key Interview Points

- **Micro-Batch Size**: Controls peak activation memory per forward/backward pass.
- **Accumulation Steps**: Multiplies micro-batch size to reach the target effective batch size.

- **Effective Batch Size**: $B_{\text{eff}} = B_{\text{micro}} \times \text{accum_steps} \times N_{\text{GPUs}}$ — what actually matters for training dynamics/learning rate scaling.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$B_{\text{eff}} = B_{\text{micro}} \times \text{accum_steps} \times N_{\text{GPUs}}$$

Peak activation memory is bounded by one micro-batch's footprint, not the effective batch's — only the accumulated gradient buffer (already sized for the full parameter count regardless of batch size) persists across accumulation steps.

Production Perspective & Trade-offs

Gradient accumulation is essentially "free" in terms of final training quality (mathematically equivalent to a true large-batch forward pass, module up to minor batch-norm-style statistics that don't apply to standard LLM training), but it costs wall-clock time — more sequential steps per optimizer update, without the throughput benefit of true large-batch parallelism.

Common Mistakes

1. Forgetting to scale the learning rate/warmup schedule based on effective batch size, not micro-batch size — LR schedules are tuned against B_{eff}.
2. Assuming gradient accumulation reduces total FLOPs — it doesn't; it trades memory for extra sequential steps, not for less total compute.

COMMON FOLLOW-UP QUESTIONS

> **Q: Does gradient accumulation change the final gradient compared to one large batch?**
>
> For standard mean-reduced losses, no — summing/averaging gradients across micro-batches is mathematically equivalent to computing the gradient on the full concatenated batch at once.

> **Q: How does gradient accumulation interact with ZeRO/FSDP?**
>
> They're complementary — accumulation controls activation memory via micro-batching, while ZeRO/FSDP controls static parameter/gradient/optimizer memory via sharding; large-scale training typically uses both together.

One-Line Takeaway

> **Takeaway:** Gradient accumulation trades wall-clock time (more sequential micro-batches) for lower peak activation memory, letting you reach a large effective batch size without needing to fit

> it all in memory at once.

---

2. Supervised Fine-Tuning (SFT) & Instruction Tuning (Q8–Q14)

Question 8: What is prompt-loss masking, and why does it matter in SFT?

[ESSENTIAL]

Conversational Answer

"Prompt-loss masking means we only compute the training loss on the response tokens, not the prompt tokens. If you trained on the full sequence — prompt plus response — the model would waste gradient signal learning to predict the instruction text itself, which is given as input at inference time, not something the model needs to generate. By zeroing out the loss on prompt positions, every gradient update is focused purely on 'given this instruction, how good was the generated response,' which is what you actually care about at deployment time."

Intuitive Example

- If the training example is `Instruction: Summarize this article. Response: <summary>`, masking ensures the loss only scores how well the model predicts the summary tokens — not how "surprising" it found the word "Instruction" or "Summarize."

Key Interview Points

- **Loss Mask**: Binary mask ($m_i \in \{0, 1\}$), 1 for response tokens, 0 for prompt tokens.
- **Gradient Focus**: No gradient flows from masked (prompt) positions — the model isn't penalized or rewarded for prompt-token predictions.
- **Denominator Matters**: The masked average divides by the *count of response tokens*, not sequence length, so loss magnitude isn't diluted by a long prompt.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{\sum_i m_i} \sum_{i=1}^{L} m_i \cdot \log P(w_i \mid w_{<i})$$

The mask ensures the denominator only counts response tokens, so the average loss reflects only the model's predictive quality on the response.

Production Perspective & Trade-offs

Without masking, longer/more complex prompts would dilute and distort the effective learning signal, and the model could learn spurious correlations from being "graded" on reproducing instruction boilerplate. This becomes especially important in multi-turn data, where the mask must cover *every* assistant turn, not just the final one.

Common Mistakes

1. Masking only the very first prompt in a multi-turn conversation and forgetting to also mask subsequent user turns — every non-assistant span needs mask = 0.
2. Confusing prompt-loss masking with padding masks — padding masks handle variable-length sequences in a batch; loss masks control *which real tokens* contribute to the loss, a separate concern.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why mask the loss on prompt tokens instead of just training on the full sequence?**
>
> Training on the full sequence wastes capacity predicting text that's given as input, not generated — masking focuses every gradient update on completion quality, which is what matters at inference.

> **Q: Does the attention mechanism also need masking, separate from the loss mask?**
>
> Yes, but for a different reason — causal attention masking (so each token only attends to previous tokens) is always required for autoregressive training, entirely independent of the loss mask that controls which positions contribute to the loss.

One-Line Takeaway

> **Takeaway:** Prompt-loss masking zeroes out the loss on instruction tokens so every gradient update is driven purely by response quality, not by how well the model predicts its own given input.

Question 9: Given token-level losses and a prompt/completion mask, how would you calculate the masked SFT loss?

[ESSENTIAL]

Conversational Answer

"I'd take the per-token negative log-likelihoods, multiply each by its mask value — so prompt positions get zeroed out entirely — then sum what's left and divide by the number of unmasked (response) tokens, not the total sequence length. That last part matters: dividing by the full sequence

length would understate the loss whenever the prompt is long relative to the response, since you'd be averaging real response-token losses against a bunch of zeros."

Intuitive Example

- For a 6-token sequence — 3 prompt tokens ("Summarize:", "cats", "sleep") and 3 response tokens ("Cats", "nap", "often") — with per-token $-\log P$ values `[2.1, 1.8, 3.0, 0.9, 1.4, 0.6]` and mask `[0, 0, 0, 1, 1, 1]`, only `0.9 + 1.4 + 0.6 = 2.9` survives masking, divided by 3 response tokens: $2.9/3 \approx 0.9667$.

Key Interview Points

- **Step 1 — Elementwise Multiply**: `per_token_loss * mask` zeroes out prompt positions entirely, not just down-weights them.
- **Step 2 — Sum Survivors**: Sum only the unmasked (response) per-token losses.
- **Step 3 — Divide by Mask Count**: Divide by `mask.sum()` (response token count), not sequence length L.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\mathcal{L}_{\text{SFT}} = \frac{\sum_i m_i \cdot (-\log P(w_i \mid w_{<i}))}{\sum_i m_i} = \frac{0.9 + 1.4 + 0.6}{3} \approx 0.9667$$

In tensor terms (`[B, L]` shaped per-token loss and mask): `masked_loss = (per_token_loss * loss_mask).sum() / loss_mask.sum().clamp(min=1)` — the `clamp(min=1)` guards against a division-by-zero edge case if a batch row has zero unmasked tokens.

Production Perspective & Trade-offs

This computation is applied per-batch, not per-example, so `loss_mask.sum()` sums across the *entire batch's* response tokens — meaning examples with longer responses naturally contribute proportionally more to the gradient than examples with short responses, unless you explicitly normalize per-example first.

Common Mistakes

1. Dividing by the full sequence length L instead of the response-token count — silently shrinks the effective loss magnitude and gradient signal whenever prompts are long.
2. Using `.view()` instead of `.reshape()` when flattening a sliced (non-contiguous) logits tensor for the cross-entropy call — raises a runtime error on slices like `logits[:, :-1, :]`.

COMMON FOLLOW-UP QUESTIONS

> **Q: What happens numerically if you forget the mask entirely?**
>
> The loss becomes a plain average over all L tokens including prompt positions — in this example, $(2.1 + 1.8 + 3.0 + 0.9 + 1.4 + 0.6)/6 \approx 1.63$, roughly 70% higher and diluted by prompt-token predictions the model was never meant to be graded on.

> **Q: Why clamp the denominator to a minimum of 1?**
>
> To avoid a division-by-zero if some pathological batch row ends up with zero unmasked (response) tokens — a defensive guard, not something that should happen in well-formed data.

One-Line Takeaway

> **Takeaway:** Masked SFT loss = (sum of per-token loss × mask) ÷ (count of unmasked response tokens) — dividing by response-token count, not total sequence length, is what makes the masking meaningful.

Question 10: How do you construct and curate a high-quality instruction dataset?

[ESSENTIAL]

Conversational Answer

"I'd think about it in three layers: coverage, quality, and diversity. Coverage means the dataset spans the range of task types and formats you expect at inference time — QA, summarization, coding, multi-turn dialogue, whatever your product needs. Quality means each response is genuinely good — well-formatted, correct, appropriately concise — because SFT data quality has an outsized effect on model behavior compared to raw pretraining data. Diversity means avoiding near-duplicate examples or narrow phrasing patterns that cause the model to overfit to a specific style rather than generalizing. In practice, this usually means combining a smaller set of high-quality human-written examples with a larger, carefully filtered synthetic set, then deduplicating and spot-checking before training."

Intuitive Example

- A dataset of 10,000 near-identical "write a poem about X" examples will teach narrow poem-writing patterns; the same 10,000 examples spread across summarization, coding, QA, and creative writing teaches broader instruction-following.

Key Interview Points

- **Coverage**: Task/format diversity matching real inference-time usage.

- **Quality over Quantity**: A smaller set of excellent examples often outperforms a much larger set of mediocre ones for SFT.
- **Deduplication**: Removing near-duplicate examples prevents overfitting to narrow phrasing patterns.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No single formula governs dataset quality, but the underlying training objective (the masked cross-entropy loss from Q8/Q9) means every low-quality or duplicated example directly shapes gradient direction — there's no implicit "averaging out" of bad examples the way there might be in a much larger pretraining corpus.

Production Perspective & Trade-offs

Curation is a recurring cost, not a one-time task — as a product's usage patterns shift, the instruction dataset needs to be re-audited for coverage gaps. Over-curating toward a narrow style also risks a "sameness" failure mode where the model produces stereotyped responses regardless of the actual instruction nuance.

Common Mistakes

1. Prioritizing raw dataset size over quality — SFT is far more sensitive to per-example quality than pretraining is.
2. Failing to check for near-duplicate examples (not just exact duplicates), which can dominate the gradient signal and cause overfitting to a narrow pattern.

COMMON FOLLOW-UP QUESTIONS

> **Q: How would you detect near-duplicate examples at scale?**
>
> Techniques like MinHash/LSH for approximate text similarity, or embedding-based clustering, to catch near-duplicates that exact string matching would miss.

> **Q: Should human-written and synthetic examples be weighted equally?**
>
> Not necessarily — many pipelines up-weight or prioritize human-written examples for quality-critical categories, using synthetic data primarily to fill coverage gaps cheaply.

One-Line Takeaway

> **Takeaway:** High-quality SFT data curation balances coverage, per-example quality, and deduplication — quality and diversity matter more for SFT than for pretraining, since every

> example directly shapes gradient direction.

Question 11: How do you handle multi-turn conversations in SFT training data?

[ESSENTIAL]

Conversational Answer

"The key idea is that the loss mask needs to cover every assistant turn in the conversation, not just the final one. You flatten the whole multi-turn exchange into one long token sequence using a chat template — special tokens marking user/assistant boundaries — and then set mask=1 on every span the assistant actually generated, across all turns, while keeping every user turn masked out. This way the model learns to produce good responses at any point in a conversation, not just as the very last reply, which matters because at inference time the model needs to handle ongoing multi-turn dialogue, not just single-shot Q&A."

Intuitive Example

- In a 3-turn conversation (user, assistant, user, assistant, user, assistant), the loss mask should be 1 across both assistant turns and 0 across all three user turns — not just 1 on the final assistant turn.

Key Interview Points

- **Chat Template**: Special tokens (`<|user|>`, `<|assistant|>`) structure the flattened conversation positionally.
- **Mask Every Assistant Turn**: Loss mask = 1 on *all* assistant spans, not just the last one.
- **Context Growth**: Later turns in the flattened sequence have progressively more context, testing the model's ability to use full conversation history.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Same masked loss formula as Q8/Q9, just applied over a longer, multi-segment mask: $m_i = 1$ for every token belonging to *any* assistant turn, $m_i = 0$ for every user turn and template token in between.

Production Perspective & Trade-offs

Multi-turn training sequences are longer, increasing both tokenization cost and activation memory per example — a trade-off against training on more single-turn examples instead. Getting the chat-

template boundaries exactly right also matters at inference time: a mismatch between training-time and inference-time template formatting silently degrades quality.

Common Mistakes

1. Only masking the final assistant turn as "the response," leaving earlier assistant turns unmasked as if they were prompt — this under-trains the model's ability to generate well mid-conversation.
2. Using a different chat template format at inference time than what was used during training — even small formatting mismatches (extra whitespace, different special tokens) can meaningfully hurt generation quality.

COMMON FOLLOW-UP QUESTIONS

> **Q: Does the attention mechanism see the full conversation history for every assistant turn?**
>
> Yes — causal attention means each assistant turn's tokens can attend to everything before it in the flattened sequence, including all prior turns, which is exactly the multi-turn context-use behavior you want to train.

> **Q: How do you handle conversations that exceed the model's context window?**
>
> Typically by truncating older turns (a sliding window) or summarizing earlier context, applied consistently between training data construction and inference-time deployment.

One-Line Takeaway

> **Takeaway:** Multi-turn SFT masks every assistant turn (not just the last) across a flattened, chat-templated sequence, teaching the model to generate well at any point in an ongoing conversation.

Question 12: What is catastrophic forgetting, and how do you mitigate it during SFT?

[ESSENTIAL]

Conversational Answer

"Catastrophic forgetting is when fine-tuning on a narrow task causes the model to lose general capabilities it had from pretraining — it 'forgets' broader knowledge or skills while overfitting to the narrow fine-tuning distribution. It happens because gradient updates on a small, narrow dataset can overwrite the weight patterns responsible for more general behavior. I'd mitigate it by keeping the fine-tuning dataset reasonably diverse rather than hyper-narrow, using a lower learning rate and fewer epochs than you might intuitively reach for, mixing in some general-instruction examples alongside

the narrow-task data, and — if using LoRA — leaning on the fact that constrained low-rank updates are inherently less likely to overwrite broad pretrained behavior than full fine-tuning."

Intuitive Example

- Fine-tuning heavily on only customer-support tickets can leave a model unable to hold a normal open-domain conversation afterward — it "forgot" general chat ability while over-specializing on support-ticket phrasing.

Key Interview Points

- **Narrow Overfitting**: Training on a narrow distribution can overwrite broader pretrained capabilities.
- **Data Mixing**: Blending general-purpose examples with task-specific data helps preserve breadth.
- **LoRA as a Natural Guard**: Constrained low-rank updates are less likely to catastrophically overwrite general behavior than unconstrained full fine-tuning.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No dedicated formula — it's an empirical training-dynamics phenomenon best diagnosed by evaluating on *held-out general capability benchmarks* before and after fine-tuning, not just on the narrow target task's own eval set (Module 08's evaluation-pitfalls discussion applies directly here).

Production Perspective & Trade-offs

Preventing forgetting by mixing in general data trades off against training efficiency (more tokens spent on data not directly related to the target task) and requires access to a reasonable general-purpose instruction set, which may not always be available for a narrowly-scoped fine-tuning project.

Common Mistakes

1. Only evaluating fine-tuned models on the narrow target task and declaring success, without checking for regressions on general capability benchmarks.
2. Assuming more training epochs always improves quality — beyond a point, additional epochs on a narrow dataset actively accelerates forgetting.

COMMON FOLLOW-UP QUESTIONS

> **Q: Does LoRA eliminate catastrophic forgetting entirely?**
>
> No, it reduces the risk (constrained low-rank updates are gentler on general behavior) but doesn't eliminate it — aggressive LoRA training on a narrow, low-diversity dataset can still cause meaningful forgetting.

> **Q: How is catastrophic forgetting related to the "alignment tax"?**
>
> They're related but distinct — the alignment tax specifically refers to capability regressions from *alignment* training (RLHF/DPO), while catastrophic forgetting is the broader phenomenon that can occur from any narrow fine-tuning, alignment-related or not.

One-Line Takeaway

> **Takeaway:** Catastrophic forgetting is narrow fine-tuning overwriting general pretrained capability — mitigate it with diverse data mixing, conservative learning rates/epochs, and preferring constrained update methods like LoRA.

Question 13: What are the risks of training on synthetic instruction data?

[ESSENTIAL]

Conversational Answer

"The biggest risks are quality drift, lack of diversity, and contamination. Quality drift means the generating model's own errors, biases, or stylistic quirks get baked into your training data — you're distilling not just knowledge but also flaws. Lack of diversity happens because LLM-generated data tends to cluster around common, high-probability phrasings unless you actively prompt for variety, so a synthetic dataset can look large in count but be narrow in actual coverage. Contamination is the sneakiest risk — if the generating prompts overlap with your evaluation set (even indirectly), you can end up training on data that leaks into your eval, inflating benchmark scores without real capability gains. I'd always run a decontamination check comparing synthetic training data against the eval set before trusting the numbers."

Intuitive Example

- Asking an LLM to generate 1,000 diverse Q&A pairs might yield mostly generic, similarly-phrased examples clustered around the model's most probable outputs — looking diverse by count, but narrow in actual semantic coverage.

Key Interview Points

- **Quality Drift**: Synthetic data inherits the generating model's own errors and biases.

- **Diversity Collapse**: LLM generations cluster around common phrasings unless explicitly steered for variety.
- **Contamination Risk**: Synthetic generation prompts can inadvertently overlap with evaluation data, inflating benchmark scores.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Decontamination is commonly checked via n-gram overlap ratio between synthetic training examples and the held-out eval set: $\text{overlap} = \frac{|\text{ngrams}(a) \cap \text{ngrams}(b)|}{|\text{ngrams}(a)|}$ — a high overlap ratio flags likely contamination requiring investigation or filtering.

Production Perspective & Trade-offs

Synthetic data is attractive because it's cheap and fast to generate at scale, but the quality-control cost (filtering, decontamination checks, diversity auditing) is real and easy to underinvest in — a large but low-quality synthetic dataset can actively hurt fine-tuning quality compared to a smaller, carefully curated one.

Common Mistakes

1. Trusting synthetic data volume as a proxy for coverage without checking actual diversity (e.g., via embedding-based clustering or simple duplicate-phrase counting).
2. Skipping decontamination checks against the eval set, leading to benchmark scores that look great but don't reflect genuine capability gains.

COMMON FOLLOW-UP QUESTIONS

> **Q: How would you improve diversity when generating synthetic instruction data?**
>
> Vary the generation prompt's topic/style constraints explicitly, use higher sampling temperature, and generate from multiple different source models or seed prompts rather than one fixed template.

> **Q: Is synthetic data ever preferable to human-written data?**
>
> Yes, for filling narrow coverage gaps cheaply (e.g., edge-case formats) where human annotation would be slow/expensive — but it's rarely a full substitute for a quality human-written core dataset.

One-Line Takeaway

> **Takeaway:** Synthetic instruction data risks inheriting the generating model's flaws, collapsing into narrow diversity, and silently contaminating evaluation sets — always run decontamination and diversity checks before trusting it.

Question 14: How do you deduplicate and decontaminate a training dataset?

[ESSENTIAL]

Conversational Answer

"Deduplication and decontamination are related but distinct passes. Deduplication removes near-identical examples *within* the training set itself, usually via exact hashing for perfect duplicates and something like MinHash/LSH or embedding similarity for near-duplicates that differ only slightly in phrasing. Decontamination compares the training set *against* the evaluation set, checking for overlap — exact matches, but also n-gram overlap for paraphrased leakage — so you can be confident a high eval score reflects genuine generalization, not memorized answers. I'd run both as an automated pipeline step before any training run, not as a one-off manual check, since contamination can silently creep back in every time the training data or eval set is updated."

Intuitive Example

- Two SFT examples with the exact same instruction but a synonym swapped in the response ("summarize" vs. "condense") would be caught by near-duplicate detection but missed by exact-match deduplication.

Key Interview Points

- **Deduplication**: Removes near-identical examples *within* the training set (exact hash + near-duplicate detection).
- **Decontamination**: Checks training data *against* the eval set for overlap, protecting the validity of benchmark scores.
- **N-gram Overlap Ratio**: A common, cheap decontamination heuristic — flags training examples sharing suspiciously many n-grams with eval examples.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{overlap}(a, b) = \frac{|\text{ngrams}_n(a) \cap \text{ngrams}_n(b)|}{|\text{ngrams}_n(a)|}$$

A threshold (e.g., overlap > 0.3) flags candidate contamination for manual review — this is a heuristic, not a perfect contamination detector, since paraphrased leakage with low n-gram overlap can still slip through.

Production Perspective & Trade-offs

Running full pairwise n-gram/embedding comparisons at scale (millions of training examples against a large eval set) can be computationally expensive — production pipelines typically use approximate methods (MinHash/LSH, approximate nearest-neighbor search) to make this tractable rather than exact pairwise comparison.

Common Mistakes

1. Only checking for exact string matches, missing paraphrased or near-duplicate contamination that changes wording but preserves the answer.
2. Treating decontamination as a one-time step rather than a required check on every data/eval-set update — contamination can silently reappear as datasets evolve.

COMMON FOLLOW-UP QUESTIONS

> **Q: What n-gram size is typically used for overlap checks?**
>
> Commonly 4-grams to 13-grams depending on the corpus — smaller n-grams catch more overlap but risk false positives from common phrases; larger n-grams are more precise but can miss shorter contaminated spans.

> **Q: How would you handle contamination found after a model has already been trained?**
>
> Flag the affected eval numbers as unreliable, remove the contaminated examples, and retrain or at minimum re-evaluate on a verified-clean eval subset before reporting results.

One-Line Takeaway

> **Takeaway:** Deduplication cleans redundancy within training data; decontamination checks training data against the eval set — both are required, automated pipeline steps to keep benchmark scores trustworthy.

---

3. Parameter-Efficient Fine-Tuning: LoRA, QLoRA, Adapters (Q15–Q21)

Question 15: How does LoRA's low-rank decomposition reduce trainable parameters?

[ESSENTIAL]

Conversational Answer

"LoRA's core insight is that the *update* you need to apply to a weight matrix during fine-tuning is usually low-rank — it doesn't need the full expressiveness of a $d \times d$ matrix to capture a task-specific adjustment. So instead of directly training the full weight matrix W, LoRA freezes W entirely and represents the update as the product of two much smaller matrices, B times A, where the inner dimension r — the rank — is tiny compared to d. You only train A and B, which together have $2rd$ parameters instead of the d^2 parameters a full update would need. Since r is typically 8 to 64 while d is in the thousands, that's a 100 to 1000x reduction in trainable parameters for that matrix."

Intuitive Example

- For a 4096×4096 weight matrix, full fine-tuning would train about 16.8 million parameters; LoRA with rank 8 trains just 65,536 — a 256x reduction, while still meaningfully adapting the layer's behavior.

Key Interview Points

- **Low-Rank Assumption**: Task-specific weight updates are assumed to lie in a low-dimensional subspace, not needing full-rank expressiveness.
- **Frozen Base + Trainable Delta**: W stays frozen; only the small A, B matrices are trained.
- **Rank r**: Controls the trade-off between parameter efficiency and expressiveness of the update.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$W' = W + \Delta W = W + BA, \qquad \text{params}_{\text{LoRA}} = 2rd \quad \text{vs.} \quad \text{params}_{\text{full}} = d^2$$

$B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times d}$, with $r \ll d$. B is initialized to zero so $\Delta W = 0$ at the start of training, guaranteeing the adapted model exactly reproduces the base model's behavior before any updates.

Production Perspective & Trade-offs

Because $\Delta W = BA$ has the exact same shape as W, it can be added back into the base weights after training with zero additional inference latency — or kept separate for multi-adapter serving. The trade-off is that a rank too low for the task's true complexity can underfit, since the update is mathematically constrained to a rank-r subspace.

Common Mistakes

1. Assuming LoRA always matches full fine-tuning quality — for tasks requiring large behavioral shifts, the rank constraint can genuinely underfit.
2. Forgetting that LoRA's parameter savings apply per adapted matrix — the total savings across a model depend on how many layers/matrices are targeted (Q20).

COMMON FOLLOW-UP QUESTIONS

> **Q: Why is B initialized to zero instead of A?**
>
> Initializing B to zero guarantees $\Delta W = BA = 0$ regardless of A's values, giving a safe, deterministic starting point; if both were zero, gradients through A would also vanish since $\partial \mathcal{L}/\partial A$ depends on B, stalling training entirely.

> **Q: Does LoRA add any inference latency?**
>
> Not if merged into the base weights post-training ($W' = W + BA$ computed once); if kept separate for multi-adapter serving, there's a small extra matmul cost per forward pass.

One-Line Takeaway

> **Takeaway:** LoRA assumes fine-tuning updates are low-rank, replacing a d^2-parameter full update with a $2rd$-parameter low-rank decomposition — a 100-1000x reduction with B zero-initialized for a safe training start.

Question 16: For a $d \times d$ layer with LoRA rank r, how many trainable parameters does LoRA introduce compared with full fine-tuning?

[ESSENTIAL]

Conversational Answer

"Full fine-tuning of a $d \times d$ matrix means training d^2 parameters. LoRA replaces that with two matrices — A of shape $r \times d$ and B of shape $d \times r$ — giving $rd + dr = 2rd$ trainable parameters total. So the reduction factor is $d^2/2rd = d/(2r)$. Plugging in real numbers — say $d = 4096$ and $r = 8$ —

full fine-tuning is 16,777,216 parameters, LoRA is $2 \times 8 \times 4096 = 65{,}536$ parameters, which works out to a 256x reduction. I'd walk through exactly that calculation live if asked, since it's the single number that best communicates why LoRA is so much cheaper."

Intuitive Example

- $d = 4096$, $r = 8$: LoRA trains 65,536 parameters versus 16,777,216 for full fine-tuning — a 256x reduction for that one matrix.

Key Interview Points

- **Full Fine-Tuning**: d^2 trainable parameters per matrix.
- **LoRA**: $2rd$ trainable parameters per matrix (A: $r \times d$, B: $d \times r$).
- **Reduction Factor**: $d^2/2rd = d/2r$ — grows larger as d increases relative to r.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{params}_{\text{LoRA}} = 2rd = 2 \times 8 \times 4096 = 65{,}536 \approx 0.066\text{M params}, \qquad \text{params}_{\text{full}} = d^2 = 4$$

$$\text{Reduction Factor} = \frac{d^2}{2rd} = \frac{d}{2r} = \frac{4096}{16} = 256\text{x}$$

Production Perspective & Trade-offs

This same $2rd$ formula scales across every targeted matrix in the model — targeting more matrices (e.g., all attention projections plus FFN layers, not just query/value) multiplies this per-matrix cost by the number of targeted matrices, which is the direct parameter-count consequence of the target-module trade-off in Q20.

Common Mistakes

1. Computing $r \times d$ only once and forgetting the factor of 2 (both A and B contribute parameters).
2. Confusing $\Delta W = BA$'s shape ($d \times d$, same as W) with the *trainable parameter count* ($2rd$, much smaller) — the update matrix's shape and its parameter count are different things.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does this reduction factor change if you double the rank?**
>
> Doubling r doubles the trainable parameter count ($2rd$ scales linearly in r), halving the reduction factor — e.g., $r = 16$ at $d = 4096$ gives 131,072 params and a 128x reduction instead of 256x.

> **Q: Does this formula change for QLoRA?**
>
> No — QLoRA uses the identical $2rd$ trainable-parameter LoRA formula; its savings come from quantizing the *frozen* base weights, not from changing the adapter's parameter count.

One-Line Takeaway

> **Takeaway:** LoRA trains $2rd$ parameters versus d^2 for full fine-tuning — a $d/2r$ reduction factor, which is 256x for the common $d = 4096$, $r = 8$ case.

Question 17: How do you choose LoRA rank (r) and alpha?

[ESSENTIAL]

Conversational Answer

"Rank controls how expressive the low-rank update can be — I'd start low, typically 8 or 16, since most tasks don't need much more, and only increase it if I'm seeing clear underfitting on a task that needs a larger behavioral shift. Alpha is a scaling factor applied to the LoRA update (ΔW is scaled by alpha/rank), and it effectively controls the update's magnitude relative to the frozen base weights — a common heuristic is setting alpha to roughly twice the rank, though this varies by task and is worth sweeping. In practice I'd treat rank as the primary capacity knob and alpha as a secondary learning-rate-like scaling knob, tuning both against a validation set rather than picking values purely by convention."

Intuitive Example

- Going from rank 8 to rank 64 on a task that's already saturating at rank 8 usually yields diminishing returns — most of the useful capacity gain happens in the lower rank range, with cost (parameters, VRAM) still increasing linearly.

Key Interview Points

- **Rank (r)**: Primary capacity knob — higher rank means a more expressive (but more expensive) update.
- **Alpha**: Scaling factor on the LoRA update magnitude, applied as $\text{scaling} = \alpha / r$.

- **Common Heuristic**: Start with $r = 8\text{-}16$ and $\alpha \approx 2r$, then sweep based on validation performance.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

The LoRA forward pass applies: $h = Wx + \text{scaling} \cdot (BA)x$, where $\text{scaling} = \alpha / r$ — so increasing α at fixed r increases the effective magnitude of the update without changing the parameter count, while increasing r increases both expressiveness and parameter count.

Production Perspective & Trade-offs

Rank directly trades off against VRAM/storage cost (linearly, via $2rd$), so the practical question is rarely "what's the best possible rank" but "what's the smallest rank that hits my quality bar" — over-provisioning rank wastes memory and mildly increases overfitting risk on small datasets without a corresponding quality gain.

Common Mistakes

1. Treating rank as a free lunch and defaulting to a high value "to be safe" — this increases cost without necessarily improving quality once the task's true complexity is exceeded.
2. Forgetting that alpha and rank interact — changing rank without re-tuning alpha's scaling ratio can unintentionally shrink or amplify the effective update magnitude.

COMMON FOLLOW-UP QUESTIONS

> **Q: What happens if alpha is set very high relative to rank?**
>
> The LoRA update gets scaled up aggressively, which can act like an effectively higher learning rate on the adapted layers — potentially causing training instability if pushed too far.

> **Q: Does the optimal rank depend on which layers you target?**
>
> Yes — attention projections and FFN layers can have different sensitivity to rank, which is part of why target-module selection (Q20) and rank selection are often tuned together, not independently.

One-Line Takeaway

> **Takeaway:** Start with a low rank (8-16) and alpha near $2r$, treating rank as the primary capacity knob and alpha as a secondary magnitude-scaling knob, tuned against validation performance rather than fixed by convention alone.

Question 18: What does QLoRA add on top of LoRA (NF4, double quantization, paged optimizers)?

[ESSENTIAL]

Conversational Answer

"QLoRA's core idea is: keep LoRA's low-rank trainable adapters exactly as they are, but quantize the *frozen* base model weights down to 4-bit NF4 instead of storing them in bf16. That alone cuts the base model's memory footprint roughly 4x. On top of that, QLoRA adds double quantization — quantizing the quantization constants themselves for a bit more savings — and paged optimizers, which use NVIDIA's unified memory to page optimizer states out to CPU RAM during momentary memory spikes instead of OOMing. Together, these are what make it possible to fine-tune a 7B+ model's LoRA adapters on a single consumer GPU with under 24GB of VRAM."

Intuitive Example

- A 7B model's frozen base weights: 14GB in bf16, but only about 3.5GB in NF4 — a 10.5GB saving that's the difference between needing a data-center GPU and fitting comfortably on a single consumer card.

Key Interview Points

- **NF4 (4-bit NormalFloat)**: Quantizes frozen base weights to ~0.5 bytes/param, roughly a 4x reduction from bf16.
- **Double Quantization**: Quantizes the quantization constants themselves for additional (smaller) memory savings.
- **Paged Optimizers**: Use unified CPU/GPU memory to page optimizer state to CPU RAM during transient memory spikes, avoiding OOM crashes.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Memory}_{\text{bf16 base}} = 2 \text{ bytes} \times 7\text{B} = 14 \text{ GB}, \qquad \text{Memory}_{\text{NF4 base}} = 0.5 \text{ bytes} \times 7\text{B} = 3.5 \text{ G}$$

The LoRA adapter's own trainable-parameter math ($2rd$) and its optimizer state are unchanged by QLoRA — quantization only touches the frozen base weights, which is why QLoRA and LoRA's parameter-efficiency numbers (Q16) are identical.

Production Perspective & Trade-offs

Quantization introduces a small but nonzero reconstruction error in the frozen base weights, which the trainable LoRA adapters (kept in full precision) must implicitly compensate for. This error can compound if a model is quantized multiple times across a pipeline (e.g., quantize → fine-tune → merge → re-quantize), which is a good reason to minimize repeated quantization round-trips.

Common Mistakes

1. Assuming QLoRA is a different training algorithm from LoRA — it's the same LoRA math, applied on top of a quantized (rather than full-precision) frozen base.
2. Forgetting that paged optimizers add latency when a page-out/page-in event actually triggers — they prevent OOM crashes, but at a real (if occasional) performance cost.

COMMON FOLLOW-UP QUESTIONS

> **Q: Does QLoRA change how many trainable parameters you have?**
>
> No — the LoRA adapter parameter count ($2rd$ per matrix) is identical; QLoRA only changes how the frozen base weights are stored.

> **Q: Why "NormalFloat" specifically, instead of a uniform 4-bit quantization?**
>
> NF4 is designed around the empirical observation that pretrained weights are roughly normally distributed, so its quantization bins are spaced to minimize error for that specific distribution rather than a general uniform range.

One-Line Takeaway

> **Takeaway:** QLoRA keeps LoRA's trainable adapters unchanged but quantizes the frozen base to 4-bit NF4 (plus double quantization and paged optimizers), cutting base memory roughly 4x and making large-model fine-tuning feasible on a single consumer GPU.

Question 19: How do Adapter layers differ from LoRA?

[ESSENTIAL]

Conversational Answer

"Adapter layers and LoRA are both parameter-efficient methods, but they insert their trainable capacity differently. Adapters add small new feed-forward bottleneck modules *in series* inside the network — typically after attention or FFN sublayers — so every forward pass has to flow *through* the adapter module sequentially, adding a bit of inference latency. LoRA instead adds its low-rank update *in parallel* alongside an existing weight matrix, computed as an additive term that can be merged back

into the original weights after training, adding zero extra inference latency once merged. That architectural difference — serial vs. parallel — is the main practical distinction; both achieve similar parameter-efficiency goals."

Intuitive Example

- An Adapter module sits like an extra checkpoint gate every request must pass through sequentially; a LoRA update is like a parallel side-calculation that gets added in and folded back into the main path, with no extra sequential hop once merged.

Key Interview Points

- **Adapters**: Inserted in series (sequentially) in the forward pass — adds inference latency, cannot be "merged away."
- **LoRA**: Inserted in parallel as an additive update — mergeable into base weights for zero extra inference latency.
- **Shared Goal**: Both dramatically reduce trainable parameters versus full fine-tuning.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Adapters compute $h' = h + f_{\text{adapter}}(h)$ as a sequential bottleneck transformation inline in the forward path; LoRA computes $h' = Wx + \text{scaling} \cdot (BA)x$ as a parallel additive term to an existing matrix multiply — the parallel structure is what enables post-training merging ($W' = W + BA$) that Adapters' sequential structure doesn't support in the same way.

Production Perspective & Trade-offs

LoRA's mergeability makes it the more common production default when a single fixed adapter configuration is deployed, since merged inference has zero overhead versus the base model. Adapters (and unmerged LoRA) are preferable when you need to swap between many task-specific modules at serving time without maintaining separate merged checkpoints per task.

Common Mistakes

1. Assuming Adapters and LoRA are interchangeable with no trade-off — the serial-vs-parallel structural difference has a real inference-latency consequence.
2. Forgetting that unmerged LoRA (kept separate for multi-adapter serving) does add a small inference cost too — the zero-overhead property only applies once merged.

COMMON FOLLOW-UP QUESTIONS

> **Q: Can Adapters be merged into the base model like LoRA?**
>
> Not as cleanly — because they're inserted as a sequential nonlinear transformation rather than a linear additive term, they generally can't be folded into the base weights the way LoRA's linear update can.

> **Q: Which is more parameter-efficient, Adapters or LoRA, for the same task?**
>
> It depends on configuration (adapter bottleneck size vs. LoRA rank), but LoRA has become the more common default in practice largely due to its mergeability and zero-overhead-at-inference property, not because it's inherently always more parameter-efficient.

One-Line Takeaway

> **Takeaway:** Adapters insert trainable modules in series (adding inference latency); LoRA adds a parallel, mergeable low-rank update (zero overhead once merged) — the structural difference, not the parameter-efficiency goal, is what distinguishes them.

Question 20: What's the trade-off in choosing which LoRA target modules to fine-tune?

[ESSENTIAL]

Conversational Answer

"The most common minimal choice is targeting just the attention projections — query and value, or all four (Q/K/V/O) — since that's usually enough capacity to adapt behavior meaningfully at a low parameter cost. Extending LoRA to also cover the feed-forward layers adds real capacity (FFN layers hold a large share of a transformer's total parameters) but multiplies your trainable parameter count and VRAM cost accordingly, since the $2rd$ formula applies per targeted matrix. So the trade-off is: fewer target modules keeps things cheap and fast but may cap quality for harder tasks; more target modules (attention plus FFN) closes that quality gap at real additional cost. I'd start narrow and only expand target modules if I see a clear quality ceiling."

Intuitive Example

- Targeting only `q_proj` / `v_proj` on a 7B model might add a few million trainable parameters total; extending to all attention projections plus FFN layers can multiply that several times over, while still remaining tiny relative to full fine-tuning's parameter count.

Key Interview Points

- **Attention-Only (Q/V or Q/K/V/O)**: Cheapest, most common default, sufficient for many narrow tasks.
- **Attention + FFN**: More capacity, proportionally more trainable parameters and VRAM cost.
- **Per-Matrix Cost**: Each additional targeted matrix adds another $2rd$ parameters, scaling total cost linearly with target-module count.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Total trainable params} = \sum_{\text{targeted matrices}} 2rd_i$$

Total LoRA parameter count is simply the sum of $2rd$ across every matrix you choose to target — so target-module selection is a direct multiplier on both parameter count and, correspondingly, optimizer-state memory for the adapters.

Production Perspective & Trade-offs

FFN layers typically hold the majority of a transformer block's parameters, so extending LoRA to cover them captures more of the model's total representational capacity — at the cost of proportionally larger adapter checkpoints and more VRAM for adapter optimizer state, though still orders of magnitude smaller than full fine-tuning.

Common Mistakes

1. Assuming attention-only targeting is always sufficient — for tasks needing substantial factual or stylistic shifts, FFN layers often matter more than attention.
2. Not accounting for the linear scaling of cost with target-module count when comparing configurations — doubling the number of targeted matrices roughly doubles the adapter's parameter and memory cost.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why are FFN layers often more parameter-dense than attention projections?**
>
> Transformer FFN blocks typically expand to 4x the hidden dimension internally, giving them a larger share of a layer's total parameters compared to the attention projections.

> **Q: Is there a standard "best" set of target modules?**
>
> No universal answer — it's task-dependent; attention-only is a strong, cheap default, with FFN inclusion reserved for tasks that show a clear quality ceiling under the narrower configuration.

One-Line Takeaway

> **Takeaway:** Each additional targeted matrix adds another $2rd$ parameters — attention-only targeting is cheap and often sufficient, while adding FFN layers closes quality gaps on harder tasks at proportionally higher cost.

Question 21: How do Prefix Tuning and Prompt Tuning differ from LoRA?

[ESSENTIAL]

Conversational Answer

"Prefix Tuning and Prompt Tuning take a completely different approach from LoRA — instead of modifying any weight matrices at all, they prepend a set of trainable 'virtual token' vectors to the input (Prompt Tuning) or to the keys/values at every attention layer (Prefix Tuning), and train only those vectors while the entire base model stays frozen and architecturally untouched. LoRA, by contrast, modifies the effective weights via a low-rank additive update. The practical consequence is that Prefix/Prompt Tuning use even fewer trainable parameters than LoRA in many configurations, but they also tend to be less expressive and can be trickier to optimize — LoRA has generally become the more popular default because it tends to match full fine-tuning quality more reliably across a wider range of tasks."

Intuitive Example

- Prompt Tuning is like prepending a few learned "hint" tokens to every input that steer the frozen model's behavior; LoRA is like giving the model itself a small, trainable behavioral adjustment at the weight level.

Key Interview Points

- **Prompt Tuning**: Trainable virtual tokens prepended to the input embeddings only.
- **Prefix Tuning**: Trainable virtual key/value vectors prepended at every attention layer.
- **LoRA**: Modifies effective weights via a low-rank additive update — generally more expressive and more reliably competitive with full fine-tuning.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Prompt/Prefix Tuning's parameter count scales with the number of virtual tokens and hidden dimension ($\text{num_tokens} \times d$, or $\times$ layers for Prefix Tuning), independent of r or the $2rd$ LoRA

formula — a structurally different parameterization rather than a variant of the same idea.

Production Perspective & Trade-offs

Because Prompt/Prefix Tuning don't modify weights at all, they can't be "merged" into the base model the way LoRA can — the virtual tokens must always be prepended at inference time, adding a small but nonzero sequence-length and compute overhead per request, similar in spirit to Adapters' serial-overhead trade-off (Q19).

Common Mistakes

1. Confusing Prompt Tuning with manual prompt engineering — Prompt Tuning learns continuous embedding vectors via gradient descent, not discrete human-readable text.
2. Assuming fewer trainable parameters always means a strictly better method — Prompt/Prefix Tuning's reduced expressiveness can mean worse quality despite the smaller parameter count.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why has LoRA become more popular than Prefix/Prompt Tuning in practice?**
>
> LoRA tends to match full fine-tuning quality more reliably across a broader range of tasks, and its mergeability gives it a zero-overhead inference path that Prefix/Prompt Tuning structurally can't offer.

> **Q: Can Prompt Tuning and LoRA be combined?**
>
> Yes — they modify different parts of the model (input embeddings vs. weight matrices) and are not mutually exclusive, though this combination is less common in practice than LoRA alone.

One-Line Takeaway

> **Takeaway:** Prefix/Prompt Tuning train virtual input tokens while leaving weights untouched, making them non-mergeable and less expressive than LoRA's weight-level low-rank update — which is why LoRA has become the more common default.

---

4. Reward Modeling & Reinforcement Learning from Human Feedback (Q22–Q26)

Question 22: How is a reward model trained using the Bradley-Terry loss?

[ESSENTIAL]

Conversational Answer

"A reward model is trained on pairwise human preference data — for a given prompt, a labeler has picked which of two candidate responses is better. The Bradley-Terry loss converts that pairwise preference into a training signal: you run both the chosen and rejected responses through the reward model to get two scalar scores, and the loss pushes the chosen response's score to be higher than the rejected one's, penalized proportionally to how wrong the current ordering is via a log-sigmoid of the score difference. Critically, this only trains the model to get the *relative ordering* right — the absolute scale of reward scores isn't directly supervised, which is fine since only relative comparisons matter for the downstream RL/DPO objective."

Intuitive Example

- Given a prompt and two candidate answers, if the reward model scores the human-preferred answer at 2.0 and the rejected one at 2.3 (backwards), the loss is high; if it scores them 3.1 and 1.2 (correctly ordered, clear margin), the loss is low.

Key Interview Points

- **Pairwise Preference Data**: Reward models train on (prompt, chosen, rejected) triples, not absolute quality labels.
- **Bradley-Terry Loss**: $-\log \sigma(r_{\text{chosen}} - r_{\text{rejected}})$ — pushes chosen score above rejected score.
- **Relative, Not Absolute**: Only the ordering/margin between chosen and rejected is supervised, not an absolute reward scale.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\mathcal{L}_{\text{RM}} = -\log \sigma(r_\theta(\text{chosen}) - r_\theta(\text{rejected})) = -F.\text{logsigmoid}(r_{\text{chosen}} - r_{\text{rejected}})$$

As the score gap $r_{\text{chosen}} - r_{\text{rejected}}$ grows more positive, $\sigma(\cdot) \to 1$ and the loss $\to 0$; as the gap goes negative (backwards ordering), the loss grows without bound.

Production Perspective & Trade-offs

Reward model quality is a hard ceiling on downstream RLHF quality — if the reward model has systematic biases or blind spots (e.g., over-rewarding verbosity), PPO will happily exploit them (reward hacking, Module 08), so reward model training data quality and coverage deserve as much scrutiny as the SFT data itself.

Common Mistakes

1. Treating reward model scores as calibrated, comparable-across-prompts absolute quality measures — they're only trained to be locally consistent within pairwise comparisons, not globally calibrated.
2. Training the reward model on too narrow or too small a preference dataset, leaving it with blind spots that PPO will later discover and exploit.

COMMON FOLLOW-UP QUESTIONS

> **Q: Does the reward model need to be as large as the policy model?**
>
> Not necessarily — reward models are often smaller than the policy model in practice, though a reward model that's too weak relative to the policy can struggle to provide a meaningful training signal.

> **Q: What happens if the human labelers disagree frequently on which response is better?**
>
> High label noise/disagreement makes the reward model's learned ordering less reliable — it's a signal to either refine labeling guidelines or treat that reward model's outputs with more caution downstream.

One-Line Takeaway

> **Takeaway:** The Bradley-Terry loss trains a scalar reward model to score the human-preferred response higher than the rejected one via a log-sigmoid of the score difference — supervising relative ordering, not an absolute reward scale.

Question 23: What is the role of the KL-divergence penalty in RLHF/PPO?

[ESSENTIAL]

Conversational Answer

"The reward model only scores individual responses in isolation — nothing about it inherently stops the policy from drifting arbitrarily far from coherent language just to chase a higher score. The KL-divergence penalty directly constrains how far the policy's output distribution is allowed to move away from a frozen reference model, usually the original SFT checkpoint. It's subtracted from the reward signal during PPO training, so the policy is optimizing for 'high reward, but don't stray too far from

what a sensible language model would produce.' Without it, PPO tends to find degenerate, reward-hacking outputs that score well but read like gibberish or exploit reward-model blind spots."

Intuitive Example

- Without the KL penalty, a policy might discover that repeating a certain phrase or padding responses with filler scores unexpectedly high on the reward model — the KL term keeps the policy anchored close enough to the reference model that this kind of degenerate drift is penalized.

Key Interview Points

- **Frozen Reference Model**: The KL penalty measures divergence against a frozen SFT copy, never updated during PPO.
- **Coherence Anchor**: Prevents the policy from drifting into degenerate, reward-hacking outputs that only look good to the reward model.
- β **Coefficient**: Controls how strongly the KL penalty constrains policy drift — tuned to stay in the "productive" reward-improvement region.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}\Big[\min\left(\rho_t \cdot \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \cdot \hat{A}_t\right)\Big] - \beta \cdot D_{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$$

β directly trades off reward-maximization against staying close to π_{ref} — as the policy is allowed to drift further (higher KL), reward model score tends to rise up to a point, then falls off a cliff as the policy starts exploiting reward-model blind spots instead of genuinely improving.

Production Perspective & Trade-offs

Tuning β is a real, ongoing production concern — too high, and the policy barely moves from the SFT starting point, wasting the RL stage's purpose; too low, and reward hacking becomes likely. Some pipelines use an adaptive KL coefficient that adjusts β dynamically to target a specific KL budget rather than fixing it upfront.

Common Mistakes

1. Treating the KL penalty as just a regularizer "for stability" without connecting it to the specific failure mode (reward hacking) it's designed to prevent.
2. Setting β once and never revisiting it — the productive KL range can shift as the reward model or dataset changes.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why is the reference model frozen rather than updated alongside the policy?**
>
> A frozen reference gives a fixed, stable anchor point — if the reference moved too, the KL penalty would be measuring drift against a constantly shifting target, undermining its purpose as a coherence constraint.

> **Q: Does DPO also use a KL-style constraint?**
>
> Yes — DPO's implicit reward reparameterization bakes an equivalent KL constraint directly into its loss function (Q27), achieving the same anchoring effect without an explicit separate reward model or RL loop.

One-Line Takeaway

> **Takeaway:** The KL penalty anchors the policy to a frozen reference model, preventing PPO from drifting into degenerate reward-hacking outputs that score well but stray far from coherent, SFT-quality language.

Question 24: What are the four model components typically involved in PPO-based RLHF, and why is their memory footprint expensive?

[ESSENTIAL]

Conversational Answer

"PPO-based RLHF needs four models resident in memory at once, each playing a distinct role. The policy model is the one actually being trained — it starts as a copy of the SFT model. The reference model is a frozen copy of that same SFT model, used only to compute the KL penalty; it never gets updated. The reward model is also frozen, and scores every response the policy generates. And the value model estimates expected future reward from a given state, which is used to compute the advantage estimate that drives the PPO update; it's trained alongside the policy. The expense comes from the fact that even though only the policy and value models are being trained, all four still need to be loaded and run inference-time forward passes on every step — so you're paying the full memory cost of four separate models simultaneously, not just the two being updated."

Intuitive Example

- For a 7B-parameter base model, even a memory-conscious PPO setup is effectively juggling four 7B-scale models' worth of memory at once — a qualitatively different infrastructure challenge than SFT or LoRA's single-model footprint.

Key Interview Points

- **Policy Model**: Trained, starts as a copy of the SFT model.

- **Reference Model**: Frozen, anchors the KL penalty against drift.
- **Reward Model**: Frozen, scores generated responses.
- **Value Model**: Trained, estimates expected future reward for the advantage calculation.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Even with PEFT reducing the *trainable* parameter count for the policy/value models, the frozen reference and reward models still consume full inference-time memory each — PEFT doesn't shrink a frozen model's footprint, only the cost of updating it. Four models means roughly $4\times$ a single model's inference memory footprint as a floor, before even counting activation memory for rollout generation.

Production Perspective & Trade-offs

This 4-model requirement is why PPO-based RLHF is often considered the most infrastructure-heavy stage of the post-pretraining pipeline — it's not just a memory-scaling problem solvable by sharding, but a qualitative increase in orchestration complexity, since four differently-roled models (two frozen, two trained) must be coordinated on every training step. This is a major reason DPO and GRPO (Q27, Q29) — which reduce or eliminate some of these models — have become popular alternatives.

Common Mistakes

1. Assuming PEFT/LoRA on the policy model solves the 4-model memory problem — it reduces the *policy's* trainable-parameter cost, but the reward and reference models are frozen and unaffected either way.
2. Forgetting the value model entirely — it's easy to remember policy/reference/reward but overlook that a separate value model is also being trained for the advantage estimate.

COMMON FOLLOW-UP QUESTIONS

> **Q: Can the reward and reference models share weights to save memory?**
>
> They're conceptually distinct (one frozen for KL, one frozen for scoring) and typically kept separate, though in some setups they may be initialized from the same checkpoint before diverging via different training histories.

> **Q: Which of the four models can be run with lower precision or offloaded most safely?**
>
> The frozen reference and reward models are good offloading/quantization candidates since they don't need gradient updates — only forward passes — making them more tolerant of precision reduction than the actively-trained policy and value models.

One-Line Takeaway

> **Takeaway:** PPO-based RLHF requires policy, reference, reward, and value models simultaneously — two frozen, two trained — making it a qualitative memory and orchestration step up from SFT or LoRA's single-model footprint.

Question 25: What causes RLHF training instability?

[ESSENTIAL]

Conversational Answer

"A few compounding factors. First, RL training is inherently higher-variance than supervised learning — you're optimizing against a noisy, sampled reward signal rather than a fixed ground-truth label. Second, a poorly tuned KL coefficient can either let the policy drift into degenerate outputs (too low) or barely let it improve at all (too high) — both look like 'instability' from a training-curve perspective, just in opposite directions. Third, the reward model itself can have blind spots or miscalibration that the policy discovers and exploits, causing reward to climb while true quality (as judged by humans) actually degrades. And finally, PPO has several interacting hyperparameters — clip range, advantage normalization, value model learning rate — any of which being off can cascade into visible instability."

Intuitive Example

- A training run where reward climbs smoothly for a while, then suddenly spikes and generation quality collapses into repetitive or nonsensical text, is a classic signature of the policy finding and exploiting a reward-model blind spot faster than the KL penalty can constrain it.

Key Interview Points

- **Sampled, Noisy Reward Signal**: RL optimizes against noisier feedback than supervised learning's fixed labels.
- **KL Coefficient Mistuning**: Too low allows degenerate drift; too high stalls improvement — both present as instability.
- **Reward Model Blind Spots**: The policy can exploit reward model miscalibration, decoupling reward score from true quality.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

PPO's clip range ϵ in $\text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)$ is specifically designed to bound how much a single update can shift the policy — but combined with a mistuned β (KL coefficient) or a noisy advantage estimate $\hat{A}_t$ from a poorly-trained value model, even a bounded per-step update can compound into visible instability over many steps.

Production Perspective & Trade-offs

Diagnosing RLHF instability in production means monitoring multiple signals together, not just reward curve — reward score alone can look great while a held-out human/LLM-as-judge eval reveals quality collapse (Module 08's divergence-detection pattern applies directly here). Catching this early requires that separate quality signal, not just the RL training's own optimized metric.

Common Mistakes

1. Monitoring only reward score as the success signal, missing reward-hacking-driven divergence between reward and true quality.
2. Treating every dip in the reward curve as instability requiring intervention — some noise is expected from RL's inherently higher-variance optimization.

COMMON FOLLOW-UP QUESTIONS

> **Q: How would you detect reward hacking early during PPO training?**
>
> Track a held-out true-quality signal (human eval samples or LLM-as-judge) alongside reward score, and watch for the two diverging — reward climbing while true quality plateaus or falls.

> **Q: Is DPO immune to this kind of instability?**
>
> DPO removes the separate reward model and the RL sampling loop entirely, which eliminates several instability sources, though it introduces its own considerations (e.g., sensitivity to the β hyperparameter and to preference data quality).

One-Line Takeaway

> **Takeaway:** RLHF instability stems from RL's inherently noisy reward signal, KL-coefficient mistuning in either direction, and reward-model blind spots the policy can exploit — all requiring a true-quality signal beyond reward score alone to detect.

Question 26: What is reward hacking, and how does it manifest during PPO training?

[ESSENTIAL]

Conversational Answer

"Reward hacking is when the policy finds a way to score highly on the reward model without genuinely producing better outputs — it's exploiting a gap between what the reward model measures and what actually constitutes quality. It manifests as a training curve where reward score keeps climbing, which looks like success, while a true-quality signal — human eval, or a separate judge model — either plateaus or actively gets worse. Classic examples include the policy learning to produce excessively long, verbose responses because the reward model has a subtle length bias, or repeating certain phrases or formatting tricks that happened to correlate with high scores in the reward model's training data without being genuinely more helpful."

Intuitive Example

- A reward model trained on data where longer, more detailed answers were often (but not causally) rated higher can inadvertently teach the policy that padding every response with extra length reliably boosts its score — reward climbs, but the responses aren't actually more helpful.

Key Interview Points

- **Reward-Quality Gap**: The policy exploits a proxy metric (reward score) diverging from the true objective (genuine response quality).
- **Reward vs. Quality Divergence**: The tell-tale signature is reward climbing while a separate true-quality signal plateaus or degrades.
- **Root Cause**: Reward model blind spots or biases (e.g., length, certain phrasings) that don't actually reflect quality.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Detection follows the same structural pattern as Module 08's divergence detector: track a proxy signal (reward score) and a true signal (held-out human/judge eval) over training steps, and flag the point where the proxy keeps improving while the true signal's trend reverses — the same "proxy improving, true quality worsening" shape as an overfitting train/val divergence, just in an RLHF context instead of supervised training.

Production Perspective & Trade-offs

The KL penalty (Q23) is the primary lever for constraining reward hacking, but it's not a complete fix — a sufficiently exploitable reward model blind spot can still be found within the allowed KL budget. The more robust (if more expensive) mitigation is continuously refreshing the reward model's training data to patch discovered blind spots as they're found.

Common Mistakes

1. Relying solely on reward score to judge RLHF training success, without a separate true-quality signal to catch divergence.
2. Assuming reward hacking is a one-time risk to check at the end of training rather than something to monitor continuously throughout.

COMMON FOLLOW-UP QUESTIONS

> **Q: How is reward hacking different from ordinary overfitting?**
>
> Structurally similar (a proxy metric improving while a true metric worsens), but overfitting is about memorizing training data, while reward hacking is specifically about exploiting a flawed proxy *objective* — the reward model itself — regardless of memorization.

> **Q: Does GRPO have the same reward hacking risk as PPO?**
>
> Yes — reward hacking is a property of optimizing against any imperfect reward signal, so it applies to GRPO too, though GRPO's group-relative advantage (Q29) changes how the signal is computed, not whether the underlying reward model can be exploited.

One-Line Takeaway

> **Takeaway:** Reward hacking is the policy exploiting a gap between the reward model's score and true response quality — detected by tracking reward alongside a separate true-quality signal and watching for divergence.

---

5. Direct Preference Optimization (DPO), GRPO & Modern Alignment (Q27–Q32)

Question 27: How does DPO eliminate the need for an explicit reward model?

[ESSENTIAL]

Conversational Answer

"DPO's key insight is that the optimal policy under the standard RLHF reward-plus-KL objective has a closed-form relationship to the reward function — which means you can algebraically substitute that relationship back into the reward model's own training loss and end up with a loss expressed purely in terms of the policy and reference model's log-probabilities, with no reward model term left at all. So instead of training a separate reward model and then running an RL loop to optimize against it, DPO directly increases the policy's relative log-probability of the preferred response over the rejected one, compared to what the frozen reference model would have assigned — the 'reward' becomes implicit in that log-probability ratio rather than a separately trained scalar function."

Intuitive Example

- Where RLHF needs a trained reward model to score "how good" a response is before running PPO against that score, DPO just directly compares how much more likely the policy makes the preferred response versus the rejected one, relative to the reference model — collapsing two stages (train reward model, then RL against it) into one supervised loss.

Key Interview Points

- **Implicit Reward**: The reward is reparameterized as the log-ratio of policy to reference probabilities — no separate model needed.
- **Single Supervised Loss**: DPO trains directly on preference pairs with one loss function, no RL rollout loop.
- **Same Underlying Objective**: DPO is mathematically derived from the same reward-plus-KL objective RLHF optimizes, just solved in closed form.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right] \right)$$

β plays the same role as the KL penalty coefficient in RLHF, controlling how far the policy is allowed to

move from the reference — but here it's baked directly into a single supervised loss rather than a separate RL penalty term.

Production Perspective & Trade-offs

Because DPO needs no reward model, no value model, and no generation rollout during training (responses are pre-collected preference pairs, not sampled live), it's substantially cheaper to run and easier to debug than PPO-based RLHF — a major reason it's become a common default. The trade-off is DPO still requires *paired* preference data, which can be harder to collect at scale than independent per-response quality ratings.

Common Mistakes

1. Thinking DPO removes the need for preference data entirely — it still requires paired (chosen, rejected) comparisons, just no separate reward model to train from them.
2. Forgetting the reference model is still required — DPO removes the *reward* model, not the reference model, which remains essential for anchoring the implicit reward.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why is the reference model still needed if there's no reward model?**
>
> The reference model anchors the implicit reward — without it, the loss would only depend on the policy's own log-probabilities, which the policy could trivially inflate for the preferred response without any grounding in what the pretrained/SFT model considered plausible.

> **Q: Does DPO require any generation/sampling during training?**
>
> No — DPO computes log-probabilities of pre-existing (chosen, rejected) response pairs; there's no autoregressive rollout generation step during training, unlike PPO or GRPO.

One-Line Takeaway

> **Takeaway:** DPO reparameterizes RLHF's reward into the log-ratio of policy-to-reference probabilities, collapsing reward model training plus RL into one supervised loss over preference pairs.

Question 28: What are the trade-offs between DPO and PPO/RLHF?

[ESSENTIAL]

Conversational Answer

"DPO is simpler, cheaper, and more stable to train — two models instead of four, no RL rollout loop, no separate reward model, and a plain supervised loss that's much easier to debug than PPO's interacting hyperparameters. The trade-off is data format and flexibility: DPO needs fixed, paired preference data collected upfront, while PPO can in principle optimize against any reward signal, including ones that come from live sampling or verifiable rewards like 'did this code pass its unit tests.' PPO is also more RL-native, which matters for tasks where exploration genuinely helps — DPO is fundamentally an offline, supervised reformulation, so it can't adapt to new preference signal the way an online RL loop can without collecting a fresh dataset."

Intuitive Example

- For aligning a chat model to human preference data collected once, DPO is usually the more practical, cheaper choice. For a coding task with a verifiable pass/fail reward from running unit tests, PPO-style RL (or GRPO) can directly optimize against that live signal in a way DPO's offline preference-pair format doesn't naturally support.

Key Interview Points

- **Model Count**: DPO needs 2 models (policy + reference); PPO needs 4 (policy, reference, reward, value).
- **Compute Profile**: DPO is compute-bound on ordinary forward/backward passes (no rollout); PPO is bound by autoregressive generation rollout.
- **Data Requirement**: DPO needs offline paired preference data; PPO can optimize against any reward signal, including live/verifiable rewards.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

DPO needs one forward pass each through the policy and reference models per preference pair — no generation rollout required at all. PPO requires autoregressive generation of responses during training (the rollout step), which is inherently sequential and far more expensive per training example than DPO's fixed-pair forward passes.

Production Perspective & Trade-offs

DPO's cost profile is much closer to SFT's than to RLHF's, making it dramatically cheaper to iterate on. PPO's flexibility to optimize against arbitrary or live reward signals (not just fixed offline pairs) is valuable specifically when the task has a natural reward function beyond human preference — e.g., verifiable correctness — which is part of why GRPO (RL-native, but without PPO's value model) has become popular for reasoning/coding tasks specifically.

Common Mistakes

1. Assuming DPO always produces equal or better results than PPO — the comparison is task-dependent; PPO/GRPO's RL formulation can better exploit verifiable or live reward signals that don't fit DPO's paired-preference format.
2. Ignoring data collection cost differences — PPO only needs a reward model (or scoring function), while DPO needs actual paired human comparisons, which can be more expensive to collect at scale.

COMMON FOLLOW-UP QUESTIONS

> **Q: If you already have a trained reward model, does that make PPO the obvious choice over DPO?**
>
> Not necessarily — you can still generate synthetic preference pairs using the reward model's rankings and train with DPO, avoiding the RL rollout loop's cost and instability while still leveraging an existing reward model.

> **Q: Which is easier to debug when training goes wrong?**
>
> DPO, generally — it's a standard supervised loss with far fewer interacting moving parts (no clip range, no value-model learning rate, no rollout sampling temperature) than PPO's RL training loop.

One-Line Takeaway

> **Takeaway:** DPO trades PPO's RL flexibility and live-reward capability for a much simpler, cheaper, two-model supervised loss — the right choice hinges on whether your reward signal is naturally offline-pairable or needs live/verifiable RL-style optimization.

Question 29: How does GRPO compute group-relative advantage without a value model?

[ESSENTIAL]

Conversational Answer

"Instead of training a separate value model to estimate 'how much better than expected was this response' — which is what PPO's advantage estimate needs — GRPO samples a *group* of multiple responses to the same prompt from the current policy, scores every one of them with the reward model, and then computes each response's advantage by normalizing its reward against that group's own mean and standard deviation. So the 'baseline' for what counts as a good response isn't a learned function at all — it's just the group's own average, recomputed fresh for every single prompt. Responses that scored above the group mean get a positive advantage and get pushed up; responses below the mean get pushed down."

Intuitive Example

- Sampling 4 responses to the same prompt with reward scores $[3.0, 5.0, 4.0, 2.0]$: the group mean is 3.5, so response 2 (reward 5.0) gets the largest positive advantage and gets reinforced the most, while response 4 (reward 2.0) gets pushed down — all computed from the group itself, with no separate value model involved.

Key Interview Points

- **Group Sampling**: G responses sampled per prompt from the current policy.
- **Group Statistics as Baseline**: Advantage = (response reward − group mean) / group std — the group replaces a learned value function.
- **No Value Model**: Eliminates the fourth model in PPO's pipeline entirely, at the cost of needing G rollouts per prompt.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$A_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$

For $r = [3.0, 5.0, 4.0, 2.0]$: mean $= 3.5$, variance $= 1.25$, std ≈ 1.118, giving advantages $A = [-0.447, 1.342, 0.447, -1.342]$ — by construction, mean-zero across the group regardless of the underlying reward values. This advantage then feeds into the same clipped PPO-style objective from Module 04, just without a value model in the loop.

Production Perspective & Trade-offs

GRPO removes the value model's training and inference cost entirely, cutting PPO's 4-model footprint down to 3 (policy, reference, reward). The trade-off is that it needs G full generation rollouts per prompt to get a meaningful group statistic — more sampling cost per training example than PPO's single-response-plus-value-model-call approach, and a small group size G makes the mean/std estimate noisy.

Common Mistakes

1. Assuming the group-relative advantage is a fixed, precomputed value like a reward model score — it's recomputed fresh per prompt, from that prompt's own sampled group.
2. Using too small a group size G and getting a noisy, unstable advantage estimate — the mean/std statistics need a reasonably sized group to be meaningful.

COMMON FOLLOW-UP QUESTIONS

> **Q: What happens if all responses in a group get the same reward?**
>
> The standard deviation is zero, which (with a small epsilon added for numerical stability) collapses all advantages toward zero — there's no relative signal to learn from when the group shows no variation.

> **Q: Why is GRPO particularly popular for reasoning/coding tasks?**
>
> Those tasks often have naturally verifiable, cheaply-computable rewards (did the code pass tests, is the final answer correct), making it easy to score a large sampled group without needing a learned reward model at all.

One-Line Takeaway

> **Takeaway:** GRPO replaces PPO's learned value model with the sampled group's own mean and standard deviation as the advantage baseline — mean-zero by construction, at the cost of needing G rollouts per prompt instead of one.

Question 30: Why does removing the critic/value model reduce GRPO's training overhead vs. PPO?

[ESSENTIAL]

Conversational Answer

"The value model in PPO isn't free — it's a fourth full model that needs its own forward passes on every training step, its own gradient updates, its own optimizer state, and its own share of GPU memory, on top of the policy, reference, and reward models. GRPO eliminates all of that by computing

the advantage baseline directly from a sampled group's statistics instead of a learned function. That cuts memory from four models down to three (policy, reference, reward), removes an entire training objective (the value model's own loss) from the pipeline, and removes one whole category of instability — a poorly-trained or high-variance value model — that could previously corrupt PPO's advantage estimates. The real trade-off is you're paying for that removed model with more generation cost instead — sampling G responses per prompt rather than one."

Intuitive Example

- PPO's advantage estimate depends on the value model being accurate; if the value model is poorly calibrated early in training, its bad estimates directly corrupt every policy update. GRPO sidesteps this failure mode entirely by never training a value model to begin with.

Key Interview Points

- **Removed Model Cost**: No value-model forward passes, gradient updates, or optimizer state — direct memory and compute savings.
- **Removed Instability Source**: A miscalibrated value model can no longer corrupt the advantage estimate, since there isn't one.
- **Shifted Cost**: The savings are partially offset by needing G rollouts per prompt for a meaningful group statistic.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Memory footprint: DPO uses 2 models (policy + reference), GRPO uses 3 (policy + reference + reward), PPO uses 4 (adds value model) — GRPO's savings versus PPO are specifically the value model's full parameter, gradient, and optimizer-state footprint, identical in kind to any other model's training cost (Module 01's 16Ψ math applies to the removed value model too).

Production Perspective & Trade-offs

Whether GRPO is actually cheaper *end-to-end* than PPO depends on the relative cost of the removed value-model training versus the added rollout cost of sampling G responses per prompt — for tasks where generation is cheap (short responses) and G can stay modest, GRPO tends to win; for tasks needing long, expensive rollouts, the extra sampling cost can partially erode the savings.

Common Mistakes

1. Assuming GRPO is unconditionally cheaper than PPO — the actual savings depend on the balance between removed value-model cost and added rollout cost for a given task.
2. Forgetting that removing the value model also removes a *source of training instability*, not just a memory cost — this is a qualitative benefit beyond the raw resource savings.

COMMON FOLLOW-UP QUESTIONS

> **Q: Does GRPO still need the reward model to be well-calibrated?**
>
> Yes — GRPO removes the value model specifically, but still relies on the reward model to produce meaningful relative scores across a sampled group; reward model quality issues (Q22, Q26) still apply.

> **Q: Could you apply the same group-relative idea to reduce PPO's other models too?**
>
> The group-relative trick specifically replaces the value model's role (estimating a baseline); it doesn't have an analogous substitute for the reward model's role (scoring responses), which both PPO and GRPO still require.

One-Line Takeaway

> **Takeaway:** Removing the value model cuts an entire model's memory/compute/instability cost from the pipeline — GRPO trades that savings against the added cost of sampling a group of G responses per prompt instead of one.

Question 31: How do IPO, KTO, ORPO, and SimPO differ from DPO?

[ESSENTIAL]

Conversational Answer

"They each tweak a different piece of DPO's formulation to address a specific limitation. IPO replaces DPO's sigmoid-log loss with a squared-loss objective, which corrects DPO's tendency to become overconfident and overfit on preference pairs that have a very clear-cut winner. KTO drops the requirement for *paired* preference data entirely — it learns from unpaired binary 'good/bad' labels on individual responses, which is useful when you only have per-response quality ratings rather than head-to-head comparisons. ORPO folds the SFT stage and the preference-alignment stage into a single combined training step, removing the need for a separate reference model altogether. And SimPO removes the reference model too, but differently — it uses length-normalized policy log-probabilities directly as the implicit reward, no reference comparison needed at all. I'd pick between them based on what data I actually have and how much I want to simplify the pipeline further."

Intuitive Example

- If you only have thumbs-up/thumbs-down ratings on individual responses (not head-to-head comparisons), KTO fits that data directly where DPO would require you to first construct pairs. If you want to skip a separate SFT stage entirely, ORPO combines both into one step.

Key Interview Points

- **IPO**: Squared-loss objective instead of sigmoid-log — corrects DPO's overconfidence on clear-cut pairs.
- **KTO**: Learns from unpaired binary good/bad labels, not paired comparisons — based on prospect theory.
- **ORPO**: Combines SFT and preference alignment into one stage, no separate reference model.
- **SimPO**: Removes the reference model, using length-normalized policy log-probabilities directly as the implicit reward.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Each method modifies one structural piece of DPO's $-\log \sigma(\beta[\ldots])$ formulation: IPO swaps the loss shape (squared vs. sigmoid-log), KTO swaps the data requirement (unpaired vs. paired), and ORPO/SimPO both remove the reference-model comparison term entirely — ORPO by merging into the SFT objective, SimPO by substituting length-normalized log-probability directly as the reward signal.

Production Perspective & Trade-offs

Removing the reference model (ORPO, SimPO) directly cuts memory back toward a single-model footprint, at the cost of losing the reference's explicit anchoring role — length normalization (SimPO) and the combined SFT+alignment objective (ORPO) are each designed to compensate for that loss in their own way, but it's a real trade-off, not a free simplification.

Common Mistakes

1. Treating all four as strictly "better than DPO" — each is a trade-off suited to specific data availability or pipeline-simplification goals, not a strict Pareto improvement.
2. Confusing KTO's unpaired-label format with DPO's paired format — they require fundamentally different data collection processes, not just a different loss function on the same data.

COMMON FOLLOW-UP QUESTIONS

> **Q: Which of these would you reach for if you only had thumbs-up/down feedback, not comparisons?**
>
> KTO — it's specifically designed for unpaired binary quality labels, which is exactly that data shape.

> **Q: Why would removing the reference model (SimPO, ORPO) be attractive?**
>
> It cuts memory back toward roughly a single model's footprint (no frozen reference copy needed), which matters most when running alignment training on large models with tight VRAM budgets.

One-Line Takeaway

> **Takeaway:** IPO, KTO, ORPO, and SimPO each relax a different constraint of DPO — loss shape, paired-data requirement, or the reference model itself — trading DPO's specific formulation for a different data-availability or memory profile.

Question 32: What data format do DPO and GRPO each require?

[ESSENTIAL]

Conversational Answer

"DPO needs offline, paired preference data — for each prompt, a chosen response and a rejected response, collected upfront, usually from human labelers or a reward model's rankings. It never samples anything live during training; it just computes log-probabilities on these fixed pairs. GRPO needs something different: it needs to sample a *group* of multiple responses to the same prompt live, from the current policy, during training — and then score each one with a reward model on the fly. So DPO's data pipeline is a one-time offline collection step, while GRPO's 'data' is generated dynamically at training time, with only the reward model (or a verifiable reward function) needing to exist upfront, not the responses themselves."

Intuitive Example

- For DPO, you'd need a dataset like `{prompt, chosen_response, rejected_response}` collected once. For GRPO, you'd need a reward function or reward model ready to score responses, and the training loop itself would generate `G` fresh responses per prompt at every step.

Key Interview Points

- **DPO**: Offline, paired `(prompt, chosen, rejected)` data — no sampling during training.
- **GRPO**: Live-sampled groups of G responses per prompt during training, scored by a reward model/function on the fly.
- **Reward Requirement**: DPO needs preference labels upfront; GRPO needs a scoring function/reward model available at training time.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

DPO's per-example cost is one forward pass each through policy and reference for a fixed pair — no generation. GRPO's per-example cost includes autoregressive generation of G responses (the same sequential, memory-bandwidth-bound decoding cost as ordinary inference) plus G reward-model scoring passes, per prompt, per training step.

Production Perspective & Trade-offs

DPO's data collection is a one-time, front-loaded cost — expensive to gather good human comparisons, but cheap to train against repeatedly afterward. GRPO's data generation is a recurring, per-step cost baked into training itself — cheaper to set up (just need a reward function, not human comparisons) but more expensive computationally every single step due to live rollout generation.

Common Mistakes

1. Assuming GRPO needs pre-collected preference data like DPO — it needs a reward *function*, not pre-collected response comparisons.
2. Assuming DPO's preference pairs must come from human labelers — they can also come from a reward model's rankings on model-generated candidates, distilling RLHF-style preference signal into DPO's offline format.

COMMON FOLLOW-UP QUESTIONS

> **Q: Could you convert GRPO-style sampled/scored data into a DPO-compatible format?**
>
> Yes — you could take the highest- and lowest-scored responses from a sampled group and treat them as a DPO (chosen, rejected) pair, effectively converting GRPO's live sampling into offline DPO training data.

> **Q: Which format is more natural for verifiable-reward tasks like math or code?**
>
> GRPO — verifiable rewards (correct/incorrect) are naturally computed live per generated response, fitting GRPO's on-the-fly scoring loop more directly than requiring pre-collected human preference pairs.

One-Line Takeaway

> **Takeaway:** DPO needs offline paired (chosen, rejected) preference data collected upfront; GRPO needs a reward function and generates its own training data live by sampling and scoring groups of responses during training.

---

6. Model Merging & Adapter Composition (Q33–Q37)

Question 33: What is "model souping" (weight averaging), and when does it work well?

[ESSENTIAL]

Conversational Answer

"Model souping is the simplest possible merge: you just directly average the weights of multiple fine-tuned models, parameter by parameter, with no task-vector reasoning involved. It works well specifically when the models being merged are all fine-tuned from the *same* base checkpoint and are reasonably similar in what they're specializing in — the classic use case is averaging several independently fine-tuned checkpoints of the same model (e.g., from different random seeds or hyperparameter runs) to get a more robust single model, often beating any individual checkpoint. It works less well when merging genuinely different tasks with conflicting weight-update directions, where naive averaging causes those directions to partially cancel rather than combine — which is exactly the problem task arithmetic and TIES/DARE are designed to handle more carefully."

Intuitive Example

- Averaging five checkpoints from five different training runs of the same fine-tuning job (same task, same base, different random seeds) is a strong "model soup" use case; averaging a coding-specialized model with a summarization-specialized model directly, with no task-vector adjustment, is exactly where naive averaging starts to hurt both specializations.

Key Interview Points

- **Same Base, Similar Task**: Model soups work best when merging checkpoints of the same task/base, not genuinely different specializations.
- **Direct Weight Average**: No task-vector subtraction — just averaging raw weights across models.
- **Failure Mode**: Conflicting task directions across genuinely different fine-tunes cause naive averaging to cancel out both specializations.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Model souping is a special case of task arithmetic where every $\lambda_i = 1/k$ and, implicitly, $\theta_{\text{base}} = 0$ (or equivalently, averaging raw weights directly rather than task vectors relative to a shared base): $\theta_{\text{merged}} = \frac{1}{k} \sum_{i=1}^{k} \theta_i$.

Production Perspective & Trade-offs

Merged models have zero additional inference cost versus any single original model — same architecture, same parameter shapes, just different values — which is the core production appeal versus serving an ensemble of separate models. The risk is silently degraded quality on tasks with conflicting merged directions, which is why validating the merged model's performance on each original task (not just assuming averaging is safe) matters before deploying it.

Common Mistakes

1. Assuming averaging always helps — it only reliably helps when merging similar/compatible fine-tunes; for genuinely conflicting tasks it can degrade both.
2. Merging models fine-tuned from *different* base checkpoints — task arithmetic and model soups both assume a shared base architecture and (ideally) shared pretrained starting weights.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why would averaging several same-task checkpoints ever beat any single one of them?**
>
> Different training runs (different seeds, data order) converge to slightly different local solutions; averaging can smooth out noise specific to any one run, similar in spirit to ensembling, without the inference-time cost of running multiple models.

> **Q: Is model souping the same as task arithmetic?**
>
> Model souping is a special, simpler case of task arithmetic — task arithmetic generalizes it by explicitly working with task vectors (differences from a shared base) and per-task coefficients, rather than averaging raw weights directly.

One-Line Takeaway

> **Takeaway:** Model souping directly averages weights across checkpoints — a strong, cheap technique when merging similar fine-tunes from the same base, but prone to canceling conflicting directions when merging genuinely different tasks.

Question 34: How does TIES-Merging resolve parameter sign conflicts across task vectors?

[ESSENTIAL]

Conversational Answer

"TIES-Merging — 'Trim, Elect Sign, Merge' — tackles the exact failure mode naive averaging has: when two task vectors disagree on the *direction* a parameter should move, a simple average partially cancels

both. TIES fixes this in three steps. First, Trim: zero out the smallest-magnitude entries in each task vector, on the assumption that tiny changes are more likely noise than meaningful specialization. Second, Elect Sign: for every remaining parameter, look across all task vectors and determine which sign — positive or negative — has the greater total magnitude support, then discard any task vector's entry at that parameter if it disagrees with the elected sign. Third, Merge: average only the surviving, sign-agreeing entries. The net effect is that conflicting-direction updates no longer cancel each other out — only entries that agree on direction get combined."

Intuitive Example

- If three fine-tuned models mostly want to increase a given parameter and one wants to decrease it, naive averaging would still shrink the net update through partial cancellation; TIES elects the majority-supported "increase" sign and simply drops the dissenting model's contribution for that parameter, instead of letting it dilute the merge.

Key Interview Points

- **Trim**: Zero out small-magnitude entries per task vector, treating them as likely noise.
- **Elect Sign**: Determine the majority-magnitude-supported sign per parameter across all task vectors.
- **Merge**: Average only entries agreeing with the elected sign — sign-disagreeing entries are excluded, not diluted.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

TIES is a pre-processing procedure applied *before* the same underlying task-arithmetic summation ($\theta_{\text{merged}} = \theta_{\text{base}} + \sum_i \lambda_i(\theta_i - \theta_{\text{base}})$) — it modifies which entries of each task vector participate in that sum and how, rather than replacing the merge equation itself.

Production Perspective & Trade-offs

TIES adds real preprocessing cost (computing per-parameter sign elections across all task vectors before merging) compared to a naive average, but directly targets the specific interference failure mode that naive merging suffers from — worthwhile whenever merging models with genuinely different, potentially conflicting specializations.

Common Mistakes

1. Thinking TIES changes the merge formula itself — it changes which entries feed into the same underlying weighted-sum merge equation, not the equation.
2. Assuming trimming small-magnitude entries always removes noise — for tasks where the true signal is genuinely small-magnitude and widespread, aggressive trimming can discard real

specialization along with noise.

COMMON FOLLOW-UP QUESTIONS

> **Q: What specific problem does the "elect sign" step solve that naive averaging doesn't?**
>
> When two task vectors disagree on the sign of a parameter's needed change, naive averaging lets them partially cancel, diluting both specializations for that parameter; TIES resolves the conflict by keeping only the entries that agree with the majority-magnitude sign, avoiding that cancellation.

> **Q: Does TIES require retraining, or is it a post-hoc merge procedure?**
>
> Purely post-hoc — it operates directly on already-trained task vectors, requiring no additional training, just the trim/elect/merge computation.

One-Line Takeaway

> **Takeaway:** TIES-Merging trims noisy small entries, elects a majority-supported sign per parameter, and merges only sign-agreeing entries — directly preventing the cancellation that naive averaging causes on conflicting task directions.

Question 35: What does DARE do differently from naive weight averaging?

[ESSENTIAL]

Conversational Answer

"DARE — Drop And REscale — takes a different angle on the same underlying interference problem. Instead of resolving sign conflicts explicitly like TIES, it exploits the observation that task vectors are highly redundant: most individual parameter changes in a fine-tuned model's task vector aren't actually necessary to preserve that task's performance. So DARE randomly drops (zeroes out) a large fraction of each task vector's entries, then rescales the surviving entries up to preserve the vector's expected overall magnitude. By sparsifying each task vector before merging, there's simply less overlap and less opportunity for conflicting entries to interfere with each other across different task vectors, which reduces the cancellation problem naive averaging suffers from — without needing an explicit sign-election step."

Intuitive Example

- If a task vector has 90% of its entries randomly dropped and the remaining 10% rescaled up by roughly 10x to preserve overall magnitude, the sparsified vector still captures most of the task's behavioral change — DARE's core empirical bet is that fine-tuned task vectors are redundant enough for this to work.

Key Interview Points

- **Random Dropping**: Zeroes out a large fraction of each task vector's entries randomly, exploiting redundancy.
- **Rescaling**: Survivors are scaled up to preserve the task vector's expected magnitude.
- **Reduced Interference**: Sparsification means fewer entries overlap across task vectors, indirectly reducing merge conflicts without explicit sign election.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Like TIES, DARE is a pre-processing step applied before the same task-arithmetic summation — it doesn't replace $\theta_{\text{merged}} = \theta_{\text{base}} + \sum_i \lambda_i(\theta_i - \theta_{\text{base}})$, it just changes which (and how many) entries of each $\theta_i - \theta_{\text{base}}$ participate, and rescales them to compensate for the dropped mass.

Production Perspective & Trade-offs

DARE's approach is stochastic (random dropping) rather than deterministic (TIES' magnitude-based trim/elect), which means results can vary run-to-run unless a fixed random seed is used — a real practical consideration when merge reproducibility matters. DARE and TIES are also not mutually exclusive; they're commonly combined (DARE's sparsification plus TIES' sign election) in modern merging pipelines.

Common Mistakes

1. Assuming DARE and TIES solve the interference problem the same way — DARE reduces interference indirectly via random sparsification/rescaling, while TIES resolves it directly via explicit sign election.
2. Forgetting the rescaling step — dropping entries without rescaling would shrink the task vector's effective magnitude, weakening the merged task's expression rather than just sparsifying it.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why does dropping most of a task vector's entries not destroy the task's performance?**
>
> Empirically, fine-tuned task vectors are highly redundant — many individual parameter changes are not independently necessary, so a large random subset (rescaled to preserve magnitude) still captures most of the task's behavioral shift.

> **Q: Can DARE and TIES be combined?**
>
> Yes — they address the same interference problem from different angles (sparsification vs. sign election) and are commonly used together in modern merging pipelines for a stronger combined effect.

One-Line Takeaway

> **Takeaway:** DARE randomly drops most of each task vector's entries and rescales the survivors to preserve magnitude, exploiting task-vector redundancy to reduce merge interference without an explicit sign-election step.

Question 36: When should you merge a LoRA adapter into the base model, and when should you keep adapters separate?

[ESSENTIAL]

Conversational Answer

"I'd merge a LoRA adapter into the base model when I'm deploying a single, fixed configuration for production — merging gives zero additional inference latency, since $W' = W + BA$ is computed once and the model behaves exactly like any ordinary dense checkpoint from then on. I'd keep adapters separate when I need to serve multiple tasks or multiple customers from one shared base model — swapping a small adapter per request is far cheaper in storage and memory than maintaining a fully merged checkpoint per task, and it lets you update or add new adapters without touching the shared base at all. The deciding factor is really: one fixed use case per deployment favors merging; many use cases sharing infrastructure favors keeping adapters separate and swappable."

Intuitive Example

- A single-purpose customer support bot with one fixed fine-tuned behavior is a good merge candidate — ship it as one dense checkpoint. A multi-tenant platform serving dozens of customers, each with their own lightweight LoRA customization on a shared base model, should keep adapters separate and load/swap them per request.

Key Interview Points

- **Merge When**: Single fixed deployment target, want zero extra inference latency, no need to swap behaviors at runtime.
- **Keep Separate When**: Multi-tenant/multi-task serving, want to swap adapters cheaply without duplicating the base model per task.
- **Storage Trade-off**: Merged checkpoints are full model size each; unmerged adapters are megabytes, shareable against one base.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Merging computes $W' = W + \text{scaling} \cdot BA$ once, producing a dense weight matrix identical in shape and inference cost to a normally fine-tuned model — the adapter's $2rd$-parameter structure disappears into the merged weights entirely, so there's no way to "un-merge" it back into a separate adapter afterward.

Production Perspective & Trade-offs

Keeping adapters separate enables patterns merging forecloses: hot-swapping adapters per request, combining multiple adapters at inference time, or rolling back a bad adapter update instantly without touching the base model. The cost is a small additional matmul at inference for the unmerged LoRA path, plus the operational complexity of routing the correct adapter to each request.

Common Mistakes

1. Merging too early in a multi-tenant setting, losing the ability to cheaply swap or roll back individual customers' adapters independently.
2. Assuming merged and unmerged LoRA always produce numerically identical outputs — they should mathematically, but implementation details (kernel fusion, precision) can introduce tiny floating-point differences worth being aware of when validating a merge.

COMMON FOLLOW-UP QUESTIONS

> **Q: Once merged, can you still separate the adapter back out later?**
>
> No — merging is a one-way operation; $W' = W + BA$ discards the individual A/B structure, so you'd need to keep the original unmerged adapter checkpoint if you might want to un-merge or update it independently later.

> **Q: Does merging change inference latency or memory versus the frozen base alone?**
>
> No — a merged model has exactly the same shape, latency, and memory footprint as any ordinary dense model of that architecture; the merge only changed weight *values*, not the computation graph.

One-Line Takeaway

> **Takeaway:** Merge LoRA adapters for a single fixed production deployment (zero extra latency); keep adapters separate for multi-tenant/multi-task serving where cheap swapping and independent updates matter more than merge simplicity.

Question 37: How would you route between multiple LoRA adapters at inference time?

[ESSENTIAL]

Conversational Answer

"The simplest approach is explicit routing at the request level — if you know which task or customer a request belongs to, you just load or select the corresponding adapter directly, no learned routing needed. Where it gets more interesting is when you want the system itself to decide which adapter (or combination of adapters) applies — that's where you'd add a lightweight routing mechanism, similar in spirit to a Mixture-of-Experts gate, that looks at the input and selects or weights which adapter(s) to apply. In production, I'd default to explicit routing whenever the task/tenant is already known from context (which is the common case), and only reach for learned routing when the system genuinely needs to infer which specialization applies purely from the input itself."

Intuitive Example

- A multi-tenant SaaS platform typically knows which customer is making a request, so it can explicitly select that customer's adapter — no learned routing needed. A single general-purpose assistant that might need coding help one moment and creative writing help the next, with no explicit task label from the user, is a better candidate for learned, input-dependent adapter routing.

Key Interview Points

- **Explicit Routing**: Select the adapter directly based on known request metadata (customer ID, task label) — simplest and most common in production.
- **Learned Routing**: A gating mechanism selects/weights adapters based on the input itself, useful when the task isn't known upfront.
- **Serving Cost**: Swapping adapters per request adds a small overhead versus a single merged model, but is far cheaper than serving separate full models per task.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Multiple adapters can also be composed rather than strictly routed — combining several adapters' outputs (a weighted sum of their BA contributions) is structurally similar to the task-arithmetic merge equation from Q33, just applied dynamically at inference time to a subset of loaded adapters rather than as a permanent offline merge.

Production Perspective & Trade-offs

Serving infrastructure for swappable adapters needs to efficiently load/unload small adapter weights per request without reloading the (much larger) frozen base model each time — this is a solved problem in most modern serving stacks (keep the base resident, hot-swap only the small adapter tensors), but it's a real infrastructure design point, not something that happens automatically.

Common Mistakes

1. Assuming adapter routing requires the same infrastructure complexity as running fully separate models per task — the base model stays shared and resident; only the small adapter needs swapping.
2. Reaching for learned/gated routing by default when explicit request-level routing (based on known metadata) would be simpler and just as effective.

COMMON FOLLOW-UP QUESTIONS

> **Q: Can you apply more than one LoRA adapter to the same request simultaneously?**
>
> Yes — this is effectively a runtime version of the merge equation, combining multiple adapters' contributions with per-adapter weights, though (as with any merge) conflicting adapters can still interfere with each other's specialization.

> **Q: Does adapter routing add meaningful inference latency?**
>
> Swapping which small adapter is active adds negligible overhead compared to the base model's own forward pass cost; the larger latency concern would be a poorly-designed learned routing mechanism adding extra compute per request.

One-Line Takeaway

> **Takeaway:** Default to explicit, metadata-based adapter routing when the task/tenant is known; reach for learned gating only when the system must infer which specialization applies purely from the input itself.

---

7. Training Production Considerations & Monitoring (Q38–Q43)

Question 38: Why do LR schedules use warmup before decay?

[ESSENTIAL]

Conversational Answer

"Starting training at the full target learning rate immediately is a common cause of early instability. Early in training, the optimizer's momentum and variance estimates haven't stabilized yet, and any newly-initialized or under-adapted components — like a fresh LoRA matrix — can take a large, destabilizing step if the learning rate is already at its peak. Linear warmup ramps the learning rate up gradually over the first chunk of training, giving the optimizer's internal statistics time to settle before the model takes its biggest steps. After warmup, a cosine decay smoothly brings the learning rate back down toward zero by the end of training, which empirically tends to produce better final convergence than either holding a constant rate or dropping it abruptly."

Intuitive Example

- Training with $\eta_{\max} = 3 \times 10^{-4}$ but no warmup often shows a visible loss spike in the first few dozen steps as the model takes an oversized initial step; adding a 100-step linear warmup to the same peak LR removes that early spike entirely.

Key Interview Points

- **Warmup**: Ramps LR linearly from 0 to peak, preventing destabilizing large early updates.
- **Cosine Decay**: Smoothly reduces LR toward zero after warmup, improving final convergence over an abrupt or constant rate.
- **Symmetry**: The cosine curve is symmetric around its midpoint, so equal LR values can appear both during warmup and decay at different training steps.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\eta(t) = \begin{cases} \eta_{\max} \cdot \dfrac{t}{T_{\text{warmup}}} & t < T_{\text{warmup}} \\ \dfrac{\eta_{\max}}{2}\left(1 + \cos\left(\pi \cdot \dfrac{t - T_{\text{warmup}}}{T_{\text{total}} - T_{\text{warmup}}}\right)\right) & t \geq T_{\text{warmup}} \end{cases}$$

For $\eta_{\max} = 3 \times 10^{-4}$, $T_{\text{warmup}} = 100$, $T_{\text{total}} = 1000$: $\eta(50) = 1.5 \times 10^{-4}$ (mid-warmup), $\eta(100) = 3.0 \times 10^{-4}$ (peak), and $\eta(550) = 1.5 \times 10^{-4}$ (mid-decay) — steps 50 and 550 land on

the same value despite being on opposite sides of the peak, a direct consequence of the cosine curve's symmetry.

Production Perspective & Trade-offs

Warmup length and peak LR both need tuning against model size and effective batch size — larger effective batch sizes (Q7) generally tolerate (and often need) a higher peak LR, but with a correspondingly longer warmup to stay stable. Skipping warmup entirely is a common, easy-to-miss cause of a bad first-few-hundred-steps loss spike that's otherwise hard to diagnose.

Common Mistakes

1. Assuming matching LR values at two different steps (e.g., 50 and 550) indicates a bug — it's expected from cosine symmetry around the midpoint.
2. Using too short a warmup for a large effective batch size, still getting early instability despite having warmup enabled at all.

COMMON FOLLOW-UP QUESTIONS

> **Q: What would you observe if warmup were skipped entirely?**
>
> A likely loss spike or instability in the first several dozen to hundred steps, as the model takes an oversized initial update before optimizer statistics have stabilized.

> **Q: Does warmup length need to scale with model size?**
>
> Generally yes — larger models and larger effective batch sizes often need proportionally longer warmup windows to avoid early instability.

One-Line Takeaway

> **Takeaway:** Warmup prevents destabilizing large early updates before optimizer statistics stabilize; cosine decay then smoothly reduces LR toward zero, together producing more stable and better-converging training than a constant or abruptly-dropped rate.

Question 39: A 3-day fine-tuning job crashes after 48 hours. How would you design checkpointing so training can resume without losing significant work?

[ESSENTIAL]

Conversational Answer

"The first thing I'd check is checkpoint frequency and completeness. Frequency-wise, I'd want checkpoints often enough that losing progress since the last one is a minor setback, not a disaster —

but not so often that the I/O cost of writing large checkpoints stalls training throughput, so I'd checkpoint asynchronously, writing to storage on a background thread while training continues on the GPU. Completeness-wise, a checkpoint has to contain more than just model weights: it needs the optimizer's internal state — Adam's momentum and variance buffers — the LR scheduler's current position, and the data-loader's position in the training set, so a resume doesn't accidentally re-train on already-seen data or skip data. If the 48-hour crash happened with checkpoints only every 12 hours, that's up to 12 hours of lost GPU-time — I'd tighten that interval and verify the resume path actually reproduces the exact same next-step loss as an uninterrupted run before trusting it in production."

Intuitive Example

- A checkpoint that only saves model weights, resumed after a crash, would restart Adam's momentum/variance from scratch — effectively re-introducing early-training-style instability partway through a long run, even though the weights themselves picked up where they left off.

Key Interview Points

- **Checkpoint Frequency**: A direct trade-off — more frequent bounds lost work on failure, but adds I/O overhead if too frequent or synchronous.
- **Full State, Not Just Weights**: Must include optimizer state, LR scheduler position, and data-loader position for exact resumption.
- **Asynchronous Writes**: Checkpointing on a background thread/process avoids stalling GPU training on I/O.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Checkpoint payload size roughly triples relative to weights alone for Adam-based training — matching the 12Ψ-byte optimizer-state figure from Module 01's 16Ψ breakdown (fp32 master weights, momentum, variance) — so checkpoint I/O cost scales accordingly, which is exactly why asynchronous, non-blocking writes matter at scale.

Production Perspective & Trade-offs

The gold-standard verification isn't just "does the checkpoint save and load without error" — it's confirming that resuming from a checkpoint produces the *same next-step loss* as an uninterrupted continuation would have. This topic's own notebook 06 demonstrated exactly this: a naive save/resume comparison showed a misleadingly large gap until both paths were run in eval mode to remove dropout's randomness as a confound, at which point resumed and uninterrupted training matched to floating-point precision.

Common Mistakes

1. Checkpointing only model weights, silently dropping optimizer state — training resumes "successfully" but with reset Adam statistics, degrading stability.
2. Checkpointing synchronously on the main training thread, causing every checkpoint write to stall GPU utilization for the full I/O duration.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why does the data-loader's position matter for exact resumption?**
>
> Without it, resuming would either re-train on data already seen before the crash (wasting an epoch's worth of signal) or skip ahead past unseen data, both subtly distorting the effective training distribution versus an uninterrupted run.

> **Q: How would you verify a checkpoint/resume implementation is actually correct, not just "doesn't crash"?**
>
> Compare the next-step loss from resuming against the next-step loss from letting the same run continue uninterrupted (in eval mode, to remove dropout randomness as a confound) — they should match to near floating-point precision if the checkpoint truly captures full state.

One-Line Takeaway

> **Takeaway:** A production checkpoint must capture model weights, optimizer state, scheduler position, and data-loader position — written asynchronously at a frequency that bounds lost work without stalling training — and should be verified by confirming exact next-step-loss reproduction on resume, not just successful loading.

Question 40: What training telemetry would you monitor to catch a failing run early?

[ESSENTIAL]

Conversational Answer

"I'd split telemetry into two categories that answer different questions. Training-time telemetry — loss curves and gradient-norm tracking — answers 'is this run proceeding normally, or has something gone numerically wrong,' and it's cheap to compute on every single step. A sudden gradient-norm spike or a loss that jumps to NaN/Inf is an immediate red flag worth automated alerting on. But training loss alone can't answer the harder question — 'is the resulting model actually good' — since a model can have excellent training loss while regressing on capabilities the training data didn't emphasize. So alongside the cheap per-step telemetry, I'd run periodic evaluation checkpoints — validation loss,

benchmark scores, and ideally a regression suite — at fixed intervals throughout the run, not just at the very end, so a regression is caught while there's still time to intervene."

Intuitive Example

- A gradient-norm plot that stays roughly flat for thousands of steps and then suddenly spikes is a classic early warning of training instability — worth catching automatically well before it visibly corrupts the loss curve.

Key Interview Points

- **Cheap, Per-Step Signals**: Loss curve and gradient-norm tracking — answers "is anything numerically wrong right now."
- **Periodic Evaluation Checkpoints**: Validation loss, benchmarks, regression suite — answers "is the model actually good," run at fixed intervals, not just at the end.
- **Automated Alerting**: NaN/Inf loss or gradient-norm spikes should trigger immediate alerts, not require a human watching a dashboard continuously.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Gradient norm is tracked as $\|\mathbf{g}\|_2$ over all trainable parameters per step — the same quantity referenced by gradient-clipping formulas — and a sustained upward trend or sudden spike is a leading indicator of instability, often visible before it shows up as a corresponding loss spike.

Production Perspective & Trade-offs

Running evaluation only at the very end of a multi-day training run means any regression discovered there costs the *entire* run to fix — running the same evaluation suite at fixed intervals throughout training catches issues while there's still budget left to adjust hyperparameters or roll back to an earlier checkpoint, at the cost of periodically pausing (or running in parallel with) training for eval compute.

Common Mistakes

1. Monitoring only training loss and declaring a run healthy, missing capability regressions training loss can't detect.
2. Only evaluating at the end of a long run, discovering a regression too late to cheaply correct it.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why isn't training loss alone sufficient to judge run health?**
>
> A model can have excellent, steadily decreasing training loss while regressing on general capabilities, safety, or benchmark-relevant tasks that the training data doesn't directly emphasize — training loss only measures next-token prediction quality on the training distribution itself.

> **Q: How would you decide how often to run the full evaluation suite versus just tracking loss/gradient norm?**
>
> Balance evaluation cost against how much compute you're willing to risk between checks — cheap per-step telemetry runs continuously, while more expensive evaluation (benchmarks, human/judge review) runs at coarser fixed intervals (e.g., every checkpoint) rather than every step.

One-Line Takeaway

> **Takeaway:** Monitor cheap per-step telemetry (loss, gradient norm) continuously for numerical red flags, and run a broader evaluation suite at fixed intervals throughout training — not just at the end — since training loss alone can't reveal capability regressions.

Question 41: What is continued pretraining / domain adaptation, and when would you use it?

[ESSENTIAL]

Conversational Answer

"Continued pretraining takes an already-pretrained model and keeps training it with the same self-supervised, next-token-prediction objective — just on a new, domain-specific corpus of raw text, rather than switching to instruction-formatted SFT data. It's the right tool when the target domain's vocabulary, style, or knowledge is meaningfully different from what the base model saw in its original pretraining — think legal documents, medical literature, or a specific codebase's conventions — and you want the model to genuinely absorb that domain's distribution before you layer SFT or alignment on top. I'd reach for it specifically when SFT alone isn't enough because the gap isn't about *following instructions well*, it's about the model not having deeply internalized the domain's language and concepts in the first place."

Intuitive Example

- Adapting a general-purpose model to a specific legal domain might start with continued pretraining on a large corpus of raw legal documents (no instruction format, just next-token prediction on legal text), *then* SFT on legal-specific instruction-following examples — the two stages address different gaps.

Key Interview Points

- **Same Objective, New Data**: Continued pretraining uses the same next-token-prediction objective as original pretraining, just on domain-specific raw text.
- **Distinct from SFT**: SFT teaches instruction-following behavior; continued pretraining teaches domain knowledge/vocabulary absorption.
- **Sequencing**: Typically continued pretraining happens *before* SFT/alignment, as a foundation those later stages build on.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No dedicated formula — it uses the identical pretraining/causal-LM loss objective from `01_llm_foundations`, just applied to a different (narrower, domain-specific) data distribution than the original pretraining corpus.

Production Perspective & Trade-offs

Continued pretraining carries the same catastrophic-forgetting risk as any full-parameter fine-tuning on a narrow distribution (Q12) — training too aggressively on a narrow domain corpus can degrade general capability, so data mixing (blending domain-specific and general text) and conservative learning rates apply here just as much as in SFT.

Common Mistakes

1. Reaching for continued pretraining when the actual gap is instruction-following behavior, not domain knowledge — that's an SFT problem, not a continued-pretraining problem.
2. Skipping data mixing and training purely on narrow domain text, risking meaningful catastrophic forgetting of general capability.

COMMON FOLLOW-UP QUESTIONS

> **Q: How is continued pretraining different from full fine-tuning on a downstream task?**
>
> The objective and data format are the difference — continued pretraining keeps the original self-supervised next-token-prediction objective on raw text, while downstream fine-tuning (SFT) typically uses instruction-formatted, loss-masked data (Q8).

> **Q: Would you use LoRA for continued pretraining, or full fine-tuning?**
>
> It depends on how large a distributional shift the target domain represents — a very large domain gap may need full fine-tuning's greater expressiveness (Q1), while a moderate shift may be well-served by LoRA at much lower cost.

One-Line Takeaway

> **Takeaway:** Continued pretraining re-applies the original next-token-prediction objective to domain-specific raw text, closing a knowledge/vocabulary gap before SFT addresses instruction-following behavior on top.

Question 42: How would you design an evaluation strategy spanning training loss, benchmarks, and human/LLM-as-judge review?

[ESSENTIAL]

Conversational Answer

"I'd think of it as layers, each catching what the others miss. Training and validation loss is the cheapest layer — it tells you if next-token prediction quality is improving on held-out data from the same distribution, but says nothing about downstream task quality or safety. Benchmark evaluation — MMLU, GSM8K, HumanEval, whatever's relevant — gives standardized, comparable task performance, but it's vulnerable to contamination and can be gamed by training on benchmark-adjacent data. Human or LLM-as-judge evaluation captures subjective quality dimensions benchmarks can't — helpfulness, tone, instruction-following nuance — but it's expensive if human, or has its own biases if it's an LLM judge. I'd run all three together, plus a fixed regression suite to catch capability drops training didn't intend, because no single layer is sufficient on its own — training loss and benchmark scores can both look healthy while a model has genuinely regressed on something none of them happen to measure."

Intuitive Example

- A model could show steadily improving training loss and stable benchmark scores while an LLM-as-judge evaluation reveals it's become noticeably more verbose or sycophantic — a regression only that third layer would catch.

Key Interview Points

- **Layered, Not Single-Metric**: No individual evaluation layer is sufficient alone — each has blind spots the others cover.
- **Training/Validation Loss**: Cheapest signal, says nothing about downstream quality or safety.
- **Benchmarks + Human/Judge + Regression Suite**: Standardized comparability, subjective quality capture, and explicit "did this get worse" checks respectively.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Layer	What it measures	Limitation
Training/validation loss	Next-token prediction quality on held-out same-distribution data	Says nothing about downstream task quality or safety
Benchmark evaluation	Standardized task performance, comparable across models	Vulnerable to contamination; gameable via benchmark-adjacent training data
Human / LLM-as-judge	Subjective quality (helpfulness, tone, instruction-following)	Expensive (human) or has its own systematic biases (LLM judge)
Regression testing	Did this run make previously-working capabilities worse	Requires maintaining a fixed regression suite across runs
Safety / contamination checks	Harmful outputs, PII leakage, benchmark leakage	Requires dedicated tooling separate from quality evaluation

Production Perspective & Trade-offs

Each layer has a different cost/frequency profile — loss and gradient telemetry run every step essentially free, benchmarks run at moderate cost on a fixed interval, and human/LLM-as-judge review is the most expensive and typically reserved for periodic checkpoints or final release gating rather than continuous monitoring.

Common Mistakes

1. Treating a single layer (often just benchmark scores) as sufficient evidence a model is "good," missing regressions that layer can't see.
2. Running evaluation with an unfixed prompt template or inconsistent harness across runs, introducing noise that masquerades as a real regression or improvement.

COMMON FOLLOW-UP QUESTIONS

> **Q: Which layer would you prioritize if compute/time were tightly constrained?**
>
> Training/validation loss and a small, fast benchmark subset as the minimum viable signal, with regression testing and human/judge review reserved for release-gating checkpoints rather than continuous monitoring.

> **Q: How do you keep LLM-as-judge evaluation from being systematically biased?**
>
> Be aware it can favor longer, more confidently-worded, or stylistically-self-similar responses regardless of actual quality — mitigations include using a different model family as judge than the one being evaluated, and periodically cross-checking judge verdicts against human review.

One-Line Takeaway

> **Takeaway:** A robust evaluation strategy layers training/validation loss, benchmarks, human/LLM-as-judge review, and regression testing together — since each layer has blind spots the others cover, and no single metric can certify a model is genuinely good.

Question 43: How do you distinguish a transient loss spike from genuine divergence during training?

[ESSENTIAL]

Conversational Answer

"A transient spike is a brief, single-step (or few-step) jump in loss that recovers on its own within a handful of subsequent steps — often caused by a particularly hard batch or a momentary numerical hiccup — and the gradient norm typically recovers alongside it. Genuine divergence looks different: loss keeps climbing (or gradient norm keeps growing) over a sustained window, without recovering, or it's paired with a corresponding drop in a held-out validation/quality signal rather than just training loss alone. I'd look at trend over a window — comparing loss (or a proxy metric) some steps back to now — rather than reacting to any single-step jump, and I'd cross-check against an independent true-quality signal where possible, since sustained divergence between a proxy improving and a true signal worsening is the real red flag, not the raw magnitude of any one spike."

Intuitive Example

- A loss curve that jumps sharply for one step then returns to its prior trend within the next few steps is almost certainly a transient hard-batch artifact; a loss curve that climbs steadily for dozens of consecutive steps without recovering, especially alongside a rising gradient norm, is a genuine divergence worth stopping the run for.

Key Interview Points

- **Transient Spike**: Brief jump that self-recovers within a few steps — usually a hard batch or minor numerical noise.
- **Genuine Divergence**: Sustained trend over a window, not recovering, often paired with true-quality regression.
- **Window-Based Detection**: Compare a metric's value now against its value several steps back, rather than reacting to any single-step change.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

A windowed trend check compares a proxy metric's direction over a fixed lookback window — e.g., is loss at step i higher than at step $i - \text{window}$ while a true-quality signal simultaneously trends the opposite direction — the same structural pattern as Module 08's reward-hacking divergence detector, just applied to ordinary training loss/gradient-norm telemetry instead of RLHF reward scores.

Production Perspective & Trade-offs

Automated alerting on raw single-step thresholds tends to be noisy (false-positives on every transient spike); windowed trend detection is more robust but adds a small detection lag — the trade-off is tuning the window size to catch real divergence quickly without drowning in false alarms from ordinary training noise.

Common Mistakes

1. Reacting to (or halting a run for) every single-step loss spike, most of which are transient and self-recovering.
2. Only watching training loss for divergence, missing the case where training loss looks fine while a held-out validation/quality signal is genuinely deteriorating (the same proxy-vs-true-quality gap seen in reward hacking).

COMMON FOLLOW-UP QUESTIONS

> **Q: What window size would you use for trend-based divergence detection?**
>
> It's a tunable trade-off — too short a window is noisy and trigger-happy on ordinary variance; too long delays detection of a real divergence; a common starting point is a window on the order of tens of steps, tuned against how noisy the specific metric is.

> **Q: If gradient norm spikes but loss doesn't, is that still worth investigating?**
>
> Yes — gradient-norm spikes are often a leading indicator that can precede a visible loss spike, so treating it as an early warning signal (rather than waiting for loss to also move) can catch instability sooner.

One-Line Takeaway

> **Takeaway:** Distinguish transient spikes (single-step, self-recovering) from genuine divergence (sustained trend over a window, ideally cross-checked against an independent true-quality signal) rather than reacting to any single-step jump in isolation.

---

8. Common Failure Modes & Best Practices (Q44–Q51)

Question 44: What is the "alignment tax," and how do you measure it?

[ESSENTIAL]

Conversational Answer

"The alignment tax is the observation that alignment training — RLHF, DPO — which optimizes hard for human preference signals, can measurably reduce performance on some capability benchmarks, particularly ones that favor concise, direct, or creative outputs. It's not a bug in any specific technique; it reflects that 'what humans rate highly in a head-to-head preference comparison' and 'what scores well on a fixed capability benchmark' are correlated but not identical objectives, so optimizing hard for one can pull slightly against the other. I'd measure it by tracking capability benchmarks *before and after* alignment training, side by side with the preference/reward metrics the alignment stage is actually optimizing — if benchmark scores dip while preference metrics improve, that gap is the alignment tax, made visible rather than left as an unmeasured trade-off."

Intuitive Example

- A model that becomes noticeably better at producing responses humans prefer in pairwise comparisons might simultaneously score slightly lower on a benchmark that rewards terse, single-fact-focused answers — the preference training nudged the model toward a style that trades against that specific benchmark's scoring criteria.

Key Interview Points

- **Not a Bug**: Reflects a genuine, expected trade-off between preference-optimization objectives and fixed benchmark objectives.
- **Measurement**: Compare capability benchmarks before/after alignment training, alongside the preference metrics being optimized.
- **Track Both, Not Either**: The fix isn't eliminating the tax, it's tracking capability benchmarks *alongside* preference metrics rather than instead of them.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No dedicated formula — it's measured empirically as $\Delta(\text{benchmark score}) = \text{score}_{\text{post-alignment}} - \text{score}_{\text{pre-alignment}}$ across a fixed capability benchmark suite, run identically before and after the alignment stage (RLHF/DPO), holding the eval harness exactly fixed to avoid confounding the measurement with the prompt-sensitivity/noise pitfalls from Q42.

Production Perspective & Trade-offs

The alignment tax is a genuine trade-off to manage, not eliminate — some benchmark regression may be an acceptable cost for meaningfully better human-perceived quality, but that trade-off should be a deliberate, measured decision, not an unmonitored side effect discovered after deployment.

Common Mistakes

1. Only tracking preference/reward metrics during alignment training and being surprised by a later-discovered benchmark regression.
2. Treating any capability benchmark dip after alignment as an outright bug to eliminate, rather than a measured, potentially acceptable trade-off against genuinely improved human-perceived quality.

COMMON FOLLOW-UP QUESTIONS

> **Q: Is the alignment tax specific to RLHF, or does it apply to DPO too?**
>
> It applies to any alignment method that optimizes hard for human/preference signals, including DPO — it's a property of the objective being optimized, not specific to PPO's RL formulation.

> **Q: How would you decide whether an alignment tax is "acceptable"?**
>
> Weigh the magnitude of benchmark regression against the size of the human-preference quality gain, and consider whether the affected benchmarks reflect capabilities your actual product depends on — a small dip on an irrelevant benchmark matters less than a dip on a benchmark tracking a core product capability.

One-Line Takeaway

> **Takeaway:** The alignment tax is preference-optimized alignment training measurably trading away some capability-benchmark performance — a real, expected trade-off to track and manage deliberately, not an unmonitored side effect.

Question 45: How does data contamination silently inflate benchmark scores?

[ESSENTIAL]

Conversational Answer

"If training data — pretraining, SFT, or preference data — overlaps with or closely resembles evaluation benchmark data, the model can end up having effectively seen (or something very close to) the answers during training, so its benchmark performance overestimates real generalization. It's called 'silent' because it's often invisible from the training loss curve alone — loss looks perfectly healthy, and benchmark scores look great, but the good scores are partly an artifact of leakage rather than genuine capability. The sneakiest version is *indirect* contamination: synthetic training data generated by a model that was itself exposed to benchmark content during its own training can leak benchmark-adjacent patterns into your dataset without you ever directly including benchmark text yourself."

Intuitive Example

- If a subset of your SFT dataset happens to closely paraphrase questions from a benchmark you're evaluating on — even without exact matches — the model can score artificially well on that benchmark without that score reflecting genuine out-of-distribution reasoning ability.

Key Interview Points

- **Invisible in Training Metrics**: Loss curves look healthy regardless of contamination — it's specifically an eval-validity issue, not a training-dynamics one.
- **Direct vs. Indirect**: Contamination can be direct (benchmark text literally in training data) or indirect (synthetic data generated by a model exposed to benchmark content).
- **Requires Dedicated Checks**: Only caught by explicit decontamination checks (n-gram overlap, embedding similarity search against the eval set), not by monitoring training itself.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Same n-gram overlap detection as Q14/Q13: $\text{overlap}(a, b) = |\text{ngrams}(a) \cap \text{ngrams}(b)| / |\text{ngrams}(a)|$, run specifically comparing training data against the *evaluation benchmark* rather than against a held-out training split — a separate check from ordinary training-set deduplication.

Production Perspective & Trade-offs

Decontamination checks need to run as a standing data-pipeline step, re-executed whenever either the training data or the evaluation benchmark set changes — contamination isn't a one-time risk to

check off, since new training data sources (especially synthetic ones) can silently reintroduce it later.

Common Mistakes

1. Assuming a clean-looking training-loss curve rules out contamination — contamination affects the *validity* of eval scores, not training dynamics, so it's invisible to loss monitoring entirely.
2. Only checking direct, exact-match contamination and missing indirect contamination introduced via synthetic data pipelines.

COMMON FOLLOW-UP QUESTIONS

> **Q: How would indirect contamination via synthetic data specifically happen?**
>
> If the model generating synthetic training data was itself exposed to benchmark content during its own pretraining, it can reproduce benchmark-adjacent patterns or even near-verbatim content in its generations, contaminating your dataset without you directly sourcing from the benchmark.

> **Q: Once contamination is discovered after training is complete, what's the fix?**
>
> Flag the affected benchmark scores as unreliable, remove or replace the contaminated training examples, and re-evaluate (or retrain) against a verified-clean benchmark subset before trusting reported numbers.

One-Line Takeaway

> **Takeaway:** Data contamination silently inflates benchmark scores by letting training data overlap — directly or indirectly via synthetic data — with evaluation content, invisible to training loss and only catchable via dedicated decontamination checks.

Question 46: What is mode collapse in the context of RLHF/DPO-trained models?

[ESSENTIAL]

Conversational Answer

"Mode collapse is when the policy converges toward a narrow set of 'safe,' high-scoring response patterns — excessive hedging, repetitive phrasing, formulaic structure — rather than genuinely diverse, high-quality outputs, because that narrow pattern reliably scores well against the reward signal being optimized. It's a close cousin of reward hacking: reward hacking is about the score-quality gap in general, while mode collapse specifically describes the resulting *homogenization* of outputs — the policy has found a local pattern that works and stopped exploring beyond it. You'd notice it by sampling many outputs across diverse prompts and seeing them converge on similar phrasing or structure, even for genuinely different underlying questions."

Intuitive Example

- A DPO-trained model that starts prefacing nearly every response with a similar hedging phrase, or a PPO-trained model whose responses to very different prompts all start converging on a near-identical sentence structure, are both classic symptoms of mode collapse — the outputs score well but have lost meaningful diversity.

Key Interview Points

- **Root Cause**: The policy converges on a narrow, high-scoring response pattern rather than exploring genuinely diverse outputs.
- **Related to Reward Hacking**: Same underlying proxy-exploitation dynamic, but specifically manifesting as output homogenization rather than any single degenerate behavior.
- **Detection**: Sample diverse prompts and check for unexpected similarity/repetition in output structure or phrasing across genuinely different questions.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No dedicated formula — diagnosed empirically via output diversity metrics (e.g., measuring lexical/structural similarity across a sample of generations on diverse prompts) rather than a training-time numeric signal; the fixes operate at the sampling/training level rather than via a corrective loss term.

Production Perspective & Trade-offs

Mitigations include diversity-aware sampling during RL (encouraging exploration beyond the narrow high-scoring pattern), a tighter KL constraint against the reference model (limiting how far the policy can drift toward any single narrow pattern), and reward model diversity checks (ensuring the reward model itself doesn't systematically favor one narrow style over genuinely varied good responses).

Common Mistakes

1. Only monitoring average reward score, missing that the score is high specifically *because* outputs have collapsed onto a narrow pattern the reward model happens to favor.
2. Assuming mode collapse only affects RL-based methods (PPO/GRPO) — DPO-trained models can exhibit the same homogenization if the preference data itself has a systematic stylistic bias.

COMMON FOLLOW-UP QUESTIONS

> **Q: How would you quantitatively detect mode collapse, not just notice it anecdotally?**
>
> Sample many generations across diverse prompts and measure output diversity directly — e.g., lexical overlap or embedding-based similarity across responses to different prompts — a sharp diversity drop after alignment training relative to the SFT baseline is a concrete signal.

> **Q: Does a tighter KL penalty always prevent mode collapse?**
>
> It helps by limiting how far the policy can drift from the reference model's more diverse baseline behavior, but it's not a complete guarantee — a narrow pattern the reference model itself already favors can still get reinforced within an otherwise-tight KL budget.

One-Line Takeaway

> **Takeaway:** Mode collapse is the policy converging on a narrow, high-scoring response pattern at the cost of output diversity — a close cousin of reward hacking, mitigated by diversity-aware sampling, KL constraints, and reward model diversity checks.

Question 47: What are common pitfalls when evaluating a fine-tuned model?

[ESSENTIAL]

Conversational Answer

"Beyond contamination specifically, there are a handful of pitfalls that make evaluation noisier or less trustworthy than it looks. Prompt sensitivity is a big one — small formatting changes in how a benchmark question is presented can swing scores substantially, so if your eval harness isn't held exactly fixed across runs, you can't trust run-to-run comparisons. High variance on small benchmark splits is another — a handful of flipped answers on a 200-question benchmark can look like a meaningful regression when it's actually within normal noise. And if you're using an LLM as a judge, that judge can have systematic biases — favoring longer, more confidently-worded, or stylistically-self-similar responses regardless of actual quality. I'd address these by fixing the eval harness exactly, using larger or multiple benchmark splits, and reporting variance alongside point estimates rather than treating any single score as ground truth."

Intuitive Example

- Comparing two training runs' benchmark scores where one used a slightly different prompt template than the other can show an apparent "regression" that's entirely an artifact of the prompt formatting change, not the model itself getting worse.

Key Interview Points

- **Prompt Sensitivity**: Small formatting changes in benchmark prompting can swing scores substantially — the eval harness must be held exactly fixed for valid comparisons.
- **Small-Split Variance**: A few flipped answers on a small benchmark can look like a real regression when it's within normal noise.
- **LLM-as-Judge Bias**: Judge models can systematically favor length, confident wording, or self-similar style over genuine quality.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No single formula, but the underlying statistical point is that small benchmark splits have wide confidence intervals around their point-estimate score — a handful of flipped answers can move the reported number well within what's statistically indistinguishable from noise, which is why reporting variance (or using a larger split) matters more than chasing a single point-estimate delta.

Production Perspective & Trade-offs

Fixing the eval harness exactly (same prompt template, same few-shot examples, same parsing logic) across every run is operationally tedious but is what makes run-to-run comparisons actually meaningful — without it, apparent regressions or improvements can be entirely artifacts of harness drift rather than real model changes.

Common Mistakes

1. Comparing benchmark scores across runs that used even slightly different prompt templates or few-shot setups, attributing the difference to the model when it's a harness artifact.
2. Treating a single-run score on a small benchmark split as a precise, noise-free measurement rather than a point estimate with real variance.

COMMON FOLLOW-UP QUESTIONS

> **Q: How would you make LLM-as-judge evaluation more trustworthy?**
>
> Use a different model family as the judge than the one being evaluated (to reduce self-similarity bias), periodically cross-check judge verdicts against human review, and be explicit about known judge biases (length, confidence) when interpreting results.

> **Q: How large does a benchmark split need to be before variance stops being a major concern?**
>
> There's no universal threshold — it depends on the benchmark's inherent score variance, but the practical takeaway is to report confidence intervals or run repeated evaluations rather than assuming any fixed split size is automatically "large enough."

One-Line Takeaway

> **Takeaway:** Trustworthy evaluation requires a fixed eval harness (avoiding prompt-sensitivity artifacts), sufficiently large benchmark splits with reported variance (avoiding noise misread as regression), and awareness of LLM-as-judge's systematic biases.

Question 48: How would you design a tight eval-training feedback loop to catch regressions early?

[ESSENTIAL]

Conversational Answer

"I'd run the layered evaluation strategy from Q42 — loss, benchmarks, regression suite, human/judge review — not just at the end of training, but at fixed checkpoint intervals throughout the run, so a regression is caught while there's still budget left to intervene, rather than discovered only after the entire multi-day job completes. Critically, I'd fix the eval harness exactly across every checkpoint (same prompts, same parsing, same judge setup) so score changes reflect real model changes, not harness drift. And I'd track a proxy-vs-true-quality pair together wherever relevant — like reward score alongside a human/judge quality sample during RLHF — specifically because the whole failure-mode catalog in this module shares the same shape: a proxy signal looking fine while the true thing you care about quietly degrades. The loop only works if it's genuinely automated and gated — a human has to actually look at the results at each checkpoint, or catching things early doesn't translate into actually stopping a bad run."

Intuitive Example

- A pipeline that automatically runs the regression suite and a benchmark subset every N training steps, alerting a human reviewer if either drops beyond a set threshold, catches a bad training trajectory at hour 10 of a 72-hour run instead of hour 72 — the difference between a minor course-correction and a wasted multi-day job.

Key Interview Points

- **Fixed-Interval, Not End-Only**: Run the full layered evaluation at checkpoints throughout training, not just at completion.
- **Fixed Harness**: Identical eval setup across every checkpoint so score changes reflect the model, not harness drift.
- **Proxy-vs-True Pairing**: Track a fast proxy signal alongside a slower true-quality signal together, watching specifically for divergence between them.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

The same windowed divergence-detection pattern from Q43 generalizes here: compare a proxy metric's trend against a true-quality metric's trend over a lookback window at each evaluation checkpoint, flagging the point where they diverge — proxy improving while true quality plateaus or worsens — as the trigger for human review or an automatic rollback.

Production Perspective & Trade-offs

A tight feedback loop trades evaluation compute/time cost against risk exposure — running the full evaluation suite more frequently catches regressions sooner but consumes more compute and (for human/judge layers) more reviewer time; the right cadence balances how expensive a late-caught regression would be against the marginal cost of more frequent checks.

Common Mistakes

1. Building the evaluation pipeline but not gating anything on it — regressions get detected in a dashboard nobody acts on before the run completes anyway.
2. Running evaluation checkpoints with a harness that isn't held fixed, introducing noise that undermines trust in the loop's regression signal.

COMMON FOLLOW-UP QUESTIONS

> **Q: What would trigger an automatic rollback versus just a human alert?**
>
> Typically, unambiguous, severe signals (NaN loss, catastrophic benchmark collapse) could trigger automatic rollback to the last good checkpoint, while more ambiguous or borderline regressions are better routed to human review rather than an automated action that might overreact to noise.

> **Q: How does this feedback loop differ for RLHF specifically versus ordinary SFT?**
>
> The core loop structure is the same, but RLHF specifically needs the reward-score-vs-true-quality pairing from Module 04/08's reward hacking discussion, since reward score alone (the RL objective) is exactly the proxy signal most likely to silently diverge from real quality.

One-Line Takeaway

> **Takeaway:** A tight eval-training feedback loop runs the layered evaluation strategy at fixed checkpoints with a held-constant harness, explicitly pairing fast proxy signals against slower true-quality signals to catch divergence — and gates real action (alerts, rollback) on what it finds.

Question 49: What's the difference between overfitting and reward hacking as failure modes?

[ESSENTIAL]

Conversational Answer

"They share the same underlying shape — a proxy metric looks like it's improving while the thing you actually care about is flat or getting worse — but they arise from different mechanisms. Overfitting is about *memorization*: the model starts fitting idiosyncrasies of the specific training examples rather than learning generalizable patterns, so training loss keeps falling while held-out validation loss stalls or rises. Reward hacking is specifically about exploiting a flawed *objective* — the reward model itself — regardless of memorization; the policy finds real, generalizable strategies that score well on the reward model's specific blind spots without genuinely being better responses. So overfitting is a data-generalization problem, while reward hacking is a proxy-objective problem — you could have a reward-hacking policy that generalizes its exploit perfectly well across new prompts, which wouldn't be true of classic overfitting."

Intuitive Example

- An overfit SFT model might memorize specific training examples' exact phrasing and fail on slightly rephrased versions of the same question. A reward-hacking RLHF policy, by contrast, might reliably produce verbose, hedge-heavy responses across *any* new prompt — a real, generalized strategy, just one that games the reward model rather than reflecting genuine quality.

Key Interview Points

- **Overfitting**: A data-generalization failure — model memorizes training-specific patterns, fails on held-out data from the same distribution.
- **Reward Hacking**: A proxy-objective failure — model finds a genuinely generalizable strategy that exploits reward model blind spots, independent of memorization.
- **Shared Detection Pattern**: Both are caught by comparing a proxy signal (training loss / reward score) against a true signal (validation loss / human-judged quality) and watching for divergence.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Both are detected via the same structural divergence check — window-compare a proxy trend against a true-signal trend and flag when they move in opposite directions (Q43's detector) — but the *root cause* differs: overfitting traces back to the training data distribution and model capacity relative to data size, while reward hacking traces back to the reward model's specific scoring function having exploitable blind spots.

Production Perspective & Trade-offs

The fixes differ accordingly: overfitting is addressed by data diversity, regularization, early stopping, or reducing model capacity relative to data (e.g., preferring PEFT over full fine-tuning); reward hacking is addressed by strengthening/updating the reward model, tightening the KL penalty, or adding independent human review — fixing one failure mode's root cause won't necessarily address the other.

Common Mistakes

1. Treating overfitting and reward hacking as the same failure mode just because they share the same "proxy improving, true signal worsening" symptom shape.
2. Applying an overfitting-style fix (e.g., more data diversity) to a reward-hacking problem, when the actual fix needed is reward-model-specific (updating it, tightening KL).

COMMON FOLLOW-UP QUESTIONS

> **Q: Can a model exhibit both failure modes simultaneously?**
>
> Yes — a model undergoing RLHF could simultaneously be overfitting to specifics of its SFT starting checkpoint's training data while also reward-hacking the RLHF reward model; they're independent failure mechanisms that can co-occur.

> **Q: Does early stopping help with reward hacking the way it helps with overfitting?**
>
> Not directly in the same way — early stopping addresses overfitting by halting before memorization dominates, but reward hacking can emerge at various points in training depending on when the policy discovers an exploitable reward model blind spot, so it needs its own dedicated monitoring (Q26) rather than relying on an overfitting-style stopping heuristic.

One-Line Takeaway

> **Takeaway:** Overfitting is a data-generalization failure (memorization); reward hacking is a proxy-objective failure (exploiting the reward model) — both share the same "proxy improving, true signal worsening" detection shape but require different root-cause fixes.

Question 50: How would you choose between full fine-tuning, LoRA/QLoRA, and RLHF/DPO for a given production use case?

[ESSENTIAL]

Conversational Answer

"I'd frame it as answering two separate questions in sequence. First: how large a *behavioral or knowledge* shift does the task need? If it's narrow — steering an already-capable model toward my

domain's format or style — LoRA is the default, escalating to QLoRA if VRAM is the binding constraint, and only reaching for full fine-tuning if the shift is large enough that PEFT demonstrably underfits. Second, and separately: does the task need *preference alignment* — teaching the model to prefer certain response styles/qualities that aren't well captured by supervised examples alone — versus just *knowledge/format* adaptation? If it's the latter, SFT (full or PEFT) is the right layer. If it's the former — you have preference data or a reward signal, and want the model to generalize a subjective quality judgment — that's when DPO (or RLHF/GRPO for a live/verifiable reward signal) comes in, typically layered *on top of* an already-SFT'd model, not as a replacement for it."

Intuitive Example

- Adapting a model to always output your company's specific JSON schema is a narrow, LoRA-appropriate SFT task. Teaching a model to be consistently more helpful and less verbose according to nuanced human preference — where "correct" isn't a single right answer — is a DPO/RLHF-appropriate alignment task, typically applied after the schema-following SFT stage, not instead of it.

Key Interview Points

- **Question 1 — Shift Size**: Narrow task → LoRA/QLoRA; large behavioral/knowledge shift → full fine-tuning.
- **Question 2 — Task Type**: Knowledge/format adaptation → SFT; subjective preference/quality alignment → DPO/RLHF/GRPO, layered on top of SFT.
- **Not Mutually Exclusive**: Real pipelines typically combine several of these in sequence (continued pretraining → SFT/LoRA → DPO), not pick just one.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Cost profile comparison: full fine-tuning carries the full 16Ψ-byte memory footprint (Q2); LoRA/QLoRA reduce trainable-parameter and (for QLoRA) base-weight cost by roughly $d/2r$ and $\sim 4\times$ respectively (Q16, Q18); DPO needs 2 models with no rollout cost, GRPO needs 3 with rollout cost, RLHF/PPO needs 4 with rollout cost (Q28, Q30) — each axis (fine-tuning method, alignment method) has an independent cost/capability trade-off to reason through.

Production Perspective & Trade-offs

The most common production pipeline shape is sequential, not either/or: continued pretraining (if domain gap is large) → SFT via LoRA or full fine-tuning (depending on shift size) → DPO or RLHF/GRPO (if preference alignment matters for the product) → possibly model merging (Q33-37) if serving multiple specialized variants. Treating these as mutually exclusive alternatives rather than a composable pipeline is a common conceptual mistake.

Common Mistakes

1. Treating full fine-tuning, PEFT, and RLHF/DPO as competing alternatives to pick exactly one of, rather than complementary stages that are often combined sequentially.
2. Reaching for RLHF/DPO when the actual gap is a knowledge/format problem better solved by SFT — alignment methods address preference/quality judgment, not missing domain knowledge.

COMMON FOLLOW-UP QUESTIONS

> **Q: Would you ever skip SFT and go straight to DPO/RLHF on a base pretrained model?**
>
> Rarely in practice — alignment methods are typically layered on top of an SFT'd (instruction-following-capable) model, since preference optimization assumes the model can already produce reasonable-quality candidate responses to differentiate between.

> **Q: How would budget constraints change this decision?**
>
> Tight budgets push toward LoRA/QLoRA over full fine-tuning, and toward DPO over RLHF/PPO (fewer models, no rollout cost) — the same shift-size and task-type reasoning still applies, just with a stronger bias toward the cheaper option at each decision point.

One-Line Takeaway

> **Takeaway:** Choose fine-tuning method by required shift size (LoRA/QLoRA for narrow, full fine-tuning for large) and choose alignment method by whether the task needs subjective preference judgment (DPO/RLHF/GRPO) versus knowledge/format adaptation (SFT alone) — and expect to combine several stages sequentially, not pick just one.

Question 51: How would you design the end-to-end post-pretraining pipeline — SFT → PEFT → alignment → merge → deploy — for a new model release?

[ESSENTIAL]

Conversational Answer

"I'd sequence it roughly like this. Start from the pretrained base and, if the target domain has a meaningful gap from the pretraining distribution, run continued pretraining first. Then SFT — using LoRA by default unless the required behavioral shift clearly demands full fine-tuning — with a curated, deduplicated, decontaminated instruction dataset and prompt-loss masking throughout, monitored with a proper LR warmup/decay schedule and checkpointing that captures full optimizer state. Next, if the product needs preference alignment beyond instruction-following, layer DPO (or GRPO/RLHF if there's a live or verifiable reward signal) on top of the SFT checkpoint, watching closely for reward-vs-quality divergence throughout. If I'm serving multiple task-specific or customer-specific variants from a

shared base, I'd use LoRA adapters throughout so I can merge or keep them separate as needed at the end — task arithmetic or TIES/DARE if merging multiple specializations, otherwise keeping adapters swappable for multi-tenant serving. Every stage gets the layered evaluation strategy — loss, benchmarks, human/judge review, regression suite — run at fixed checkpoints, not just at the end, with automated decontamination and divergence checks wired in throughout, not bolted on afterward."

Intuitive Example

- A concrete pipeline: raw domain corpus → continued pretraining (if needed) → LoRA SFT on a curated, decontaminated instruction dataset → DPO on human preference pairs collected against the SFT checkpoint → evaluate against the layered strategy at each stage → merge adapters (or keep separate for multi-tenant serving) → final release-gate evaluation before deployment.

Key Interview Points

- **Sequential, Composable Stages**: Continued pretraining → SFT (LoRA/full) → alignment (DPO/RLHF/GRPO) → merge/serve — not independent alternatives.
- **Evaluation at Every Stage**: The layered evaluation strategy (Q42) runs at each stage's checkpoints, not just at final release.
- **Data Hygiene Throughout**: Deduplication and decontamination apply at every data-touching stage — SFT data, preference data, and any synthetic data used along the way.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Each pipeline stage's cost stacks: continued pretraining and full-parameter SFT carry the full 16Ψ memory profile (Q2-3); LoRA-based SFT cuts that to a $2rd$-parameter adapter cost (Q16); DPO adds a 2-model cost with no rollout (Q28); RLHF/GRPO add rollout-bound generation cost with 3-4 models (Q24, Q30) — the total pipeline cost is the sum of whichever stages are actually included, which is exactly why matching each stage to genuine need (not including a stage "just in case") matters for a production budget.

Production Perspective & Trade-offs

The biggest practical risk in a multi-stage pipeline isn't any single stage failing outright — it's a silent regression introduced at an early stage (e.g., contamination in the SFT data) propagating invisibly through every subsequent stage until it surfaces at final evaluation, by which point diagnosing which stage introduced it is much harder. This is the core argument for evaluating at every stage boundary, not just at the end.

Common Mistakes

1. Only running the full evaluation suite at the very end of the pipeline, making it hard to localize which stage introduced a discovered regression.
2. Skipping decontamination checks at intermediate stages (e.g., only checking the final SFT dataset, not preference data or synthetic data used later), letting contamination slip in downstream.

COMMON FOLLOW-UP QUESTIONS

> **Q: How would you decide which stages to skip for a smaller, lower-budget release?**
>
> Continued pretraining and RLHF/PPO are the most skippable under budget pressure (domain gap permitting, and preferring DPO's cheaper 2-model alternative to full RLHF) — SFT and basic decontamination/evaluation hygiene are the hardest stages to skip without real quality risk.

> **Q: Where would model merging (Q33-37) fit into this pipeline if you're serving multiple product variants?**
>
> Typically at the very end, after each variant has gone through its own SFT/alignment stages as a LoRA adapter on the shared base — merge them via task arithmetic/TIES/DARE if you want one combined model, or keep them separate and swappable if you need per-variant serving.

One-Line Takeaway

> **Takeaway:** Sequence continued pretraining, SFT (LoRA-first), alignment (DPO/RLHF/GRPO as needed), and merging/serving as composable stages — each matched to genuine need rather than included by default — with the layered evaluation and decontamination checks running at every stage boundary, not just at the end.

LLM Fine-tuning & Alignment Interview Cheatsheet: Final Revision Sheet

Quick-Recall One-Line Takeaways Table

#	Question	One-Line Takeaway
1	Full FT vs. PEFT	Default to PEFT for narrow, steerable tasks; reach for full fine-tuning only when the task demands a genuinely large shift in model behavior and you have the budget to match.

#	Question	One-Line Takeaway
2	Why Adam dominates VRAM	Adam's fp32 master weights plus momentum and variance buffers add up to 12Ψ bytes — three-quarters of the total 16Ψ training memory footprint.
3	7B full-FT VRAM estimate	Estimate full fine-tuning memory as a fixed 16Ψ-byte static cost (112GB for a 7B model) plus a batch/sequence-dependent activation cost.
4	DP vs. TP vs. PP	Data Parallelism replicates the model and splits data; Tensor Parallelism splits individual layer computations; Pipeline Parallelism splits the model by layer across GPUs.
5	ZeRO stages 1-3	ZeRO-1 shards optimizer states, ZeRO-2 adds gradients, ZeRO-3 adds parameters — each stage trades more communication for a larger per-GPU memory reduction.
6	FSDP vs. ZeRO	FSDP and ZeRO-3 shard memory the same way — the real choice is ecosystem fit, not a difference in the underlying algorithm.
7	Micro-batch vs. accumulation	Gradient accumulation trades wall-clock time for lower peak activation memory, letting you reach a large effective batch size without needing it all in memory at once.
8	Prompt-loss masking	Prompt-loss masking zeroes out the loss on instruction tokens so every gradient update is driven purely by response quality.
9	Masked SFT loss calculation	Masked SFT loss = (sum of per-token loss × mask) ÷ (count of unmasked response tokens), not total sequence length.
10	Instruction dataset curation	High-quality SFT data curation balances coverage, per-example quality, and deduplication — quality matters more for SFT than for pretraining.
11	Multi-turn SFT masking	Multi-turn SFT masks every assistant turn (not just the last) across a flattened, chat-templated sequence.
12	Catastrophic forgetting	Catastrophic forgetting is narrow fine-tuning overwriting general pretrained capability — mitigate with data mixing, conservative LR/epochs, and LoRA.
13	Synthetic data risks	Synthetic instruction data risks inheriting the generating model's flaws, collapsing into narrow diversity, and silently contaminating

#	Question	One-Line Takeaway
		evaluation sets.
14	Dedup vs. decontamination	Deduplication cleans redundancy within training data; decontamination checks training data against the eval set — both are required.
15	LoRA low-rank decomposition	LoRA assumes fine-tuning updates are low-rank, replacing a d^2-parameter full update with a $2rd$-parameter decomposition.
16	LoRA param-count calculation	LoRA trains $2rd$ parameters versus d^2 for full fine-tuning — a $d/2r$ reduction factor, 256x for $d = 4096, r = 8$.
17	Choosing rank & alpha	Start with a low rank (8-16) and alpha near $2r$, treating rank as the capacity knob and alpha as the magnitude-scaling knob.
18	QLoRA additions	QLoRA quantizes the frozen base to 4-bit NF4 (plus double quantization and paged optimizers), cutting base memory roughly 4x.
19	Adapters vs. LoRA	Adapters insert trainable modules in series (adding latency); LoRA adds a parallel, mergeable low-rank update (zero overhead once merged).
20	LoRA target module trade-off	Each additional targeted matrix adds another $2rd$ parameters — attention-only is cheap and often sufficient; adding FFN closes quality gaps at higher cost.
21	Prefix/Prompt Tuning vs. LoRA	Prefix/Prompt Tuning train virtual input tokens while leaving weights untouched, making them non-mergeable and less expressive than LoRA.
22	Bradley-Terry reward training	The Bradley-Terry loss trains a scalar reward model to score the preferred response higher than the rejected one via log-sigmoid of the score difference.
23	KL penalty role in RLHF	The KL penalty anchors the policy to a frozen reference model, preventing PPO from drifting into degenerate reward-hacking outputs.
24	Four RLHF models	PPO-based RLHF requires policy, reference, reward, and value models simultaneously — two frozen, two trained.

#	Question	One-Line Takeaway
25	RLHF instability causes	RLHF instability stems from RL's noisy reward signal, KL-coefficient mistuning, and reward-model blind spots the policy can exploit.
26	Reward hacking	Reward hacking is the policy exploiting a gap between the reward model's score and true response quality.
27	DPO's implicit reward	DPO reparameterizes RLHF's reward into the log-ratio of policy-to-reference probabilities, collapsing reward model training plus RL into one supervised loss.
28	DPO vs. PPO trade-offs	DPO trades PPO's RL flexibility and live-reward capability for a much simpler, cheaper, two-model supervised loss.
29	GRPO group-relative advantage	GRPO replaces PPO's learned value model with the sampled group's own mean and standard deviation as the advantage baseline.
30	Why GRPO cuts overhead	Removing the value model cuts an entire model's memory/compute/instability cost — GRPO trades that against the added cost of sampling a group.
31	IPO/KTO/ORPO/SimPO	Each relaxes a different constraint of DPO — loss shape, paired-data requirement, or the reference model itself.
32	DPO vs. GRPO data format	DPO needs offline paired preference data collected upfront; GRPO generates its own training data live by sampling and scoring groups.
33	Model souping	Model souping directly averages weights across checkpoints — strong when merging similar fine-tunes, prone to canceling conflicting directions otherwise.
34	TIES-Merging	TIES-Merging trims noisy entries, elects a majority-supported sign per parameter, and merges only sign-agreeing entries.
35	DARE	DARE randomly drops most of each task vector's entries and rescales survivors, exploiting redundancy to reduce merge interference.
36	Merge vs. keep adapters separate	Merge LoRA adapters for a single fixed deployment (zero extra latency); keep adapters separate for multi-tenant serving.

#	Question	One-Line Takeaway
37	Multi-adapter routing	Default to explicit, metadata-based adapter routing when the task/tenant is known; reach for learned gating only when necessary.
38	LR warmup + decay	Warmup prevents destabilizing large early updates; cosine decay then smoothly reduces LR, producing more stable, better-converging training.
39	Production checkpointing	A production checkpoint must capture weights, optimizer state, scheduler position, and data-loader position — written asynchronously and resume-verified.
40	Training telemetry	Monitor cheap per-step telemetry (loss, gradient norm) continuously, and run a broader evaluation suite at fixed intervals, not just at the end.
41	Continued pretraining	Continued pretraining re-applies the next-token-prediction objective to domain-specific raw text, closing a knowledge gap before SFT.
42	Layered evaluation strategy	A robust evaluation strategy layers loss, benchmarks, human/LLM-as-judge review, and regression testing — no single metric is sufficient alone.
43	Loss spike vs. divergence	Distinguish transient spikes (self-recovering) from genuine divergence (sustained trend, cross-checked against a true-quality signal).
44	Alignment tax	The alignment tax is preference-optimized alignment training measurably trading away some capability-benchmark performance.
45	Data contamination	Data contamination silently inflates benchmark scores by letting training data overlap — directly or indirectly — with evaluation content.
46	Mode collapse	Mode collapse is the policy converging on a narrow, high-scoring response pattern at the cost of output diversity.
47	Evaluation pitfalls	Trustworthy evaluation requires a fixed eval harness, sufficiently large benchmark splits with reported variance, and awareness of judge bias.

#	Question	One-Line Takeaway
48	Eval-training feedback loop	A tight feedback loop runs layered evaluation at fixed checkpoints with a held-constant harness, gating real action on what it finds.
49	Overfitting vs. reward hacking	Overfitting is a data-generalization failure (memorization); reward hacking is a proxy-objective failure (exploiting the reward model).
50	Choosing FT/PEFT/alignment method	Choose fine-tuning method by required shift size, alignment method by whether the task needs subjective preference judgment.
51	End-to-end pipeline design	Sequence continued pretraining, SFT (LoRA-first), alignment, and merging as composable stages, each matched to genuine need.

Essential Formula Cheat Sheet

Mixed-Precision Training Memory (Module 01):

$$\text{Memory}_{\text{bytes}} = \underbrace{2\Psi}_{\text{fp16 params}} + \underbrace{2\Psi}_{\text{fp16 grads}} + \underbrace{4\Psi + 4\Psi + 4\Psi}_{\text{fp32 master + Adam } m,v} = 16\Psi \text{ bytes}$$

ZeRO Stage Memory:

$$\text{Memory}_{\text{ZeRO-1}} = 2\Psi + 2\Psi + \frac{12\Psi}{N}, \quad \text{Memory}_{\text{ZeRO-2}} = 2\Psi + \frac{14\Psi}{N}, \quad \text{Memory}_{\text{ZeRO-3}} = \frac{16}{N}$$

Effective Batch Size:

$$B_{\text{eff}} = B_{\text{micro}} \times \text{accum_steps} \times N_{\text{GPUs}}$$

Masked SFT Loss:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{\sum_i m_i} \sum_{i=1}^{L} m_i \cdot \log P(w_i \mid w_{<i})$$

LoRA Parameter Count:

$$W' = W + BA, \qquad \text{params}_{\text{LoRA}} = 2rd \quad \text{vs.} \quad \text{params}_{\text{full}} = d^2$$

QLoRA NF4 Base Storage:

$$\text{Memory}_{\text{bf16}} = 2 \text{ bytes/param}, \qquad \text{Memory}_{\text{NF4}} = 0.5 \text{ bytes/param}$$

Bradley-Terry Reward Loss:

$$\mathcal{L}_{\text{RM}} = -\log \sigma(r_\theta(\text{chosen}) - r_\theta(\text{rejected}))$$

PPO Clipped Objective with KL Penalty:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}\Big[\min\left(\rho_t \cdot \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \cdot \hat{A}_t\right)\Big] - \beta \cdot D_{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$$

DPO Loss:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta\left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right]\right)$$

GRPO Group-Relative Advantage:

$$A_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$

Task Arithmetic Merge:

$$\theta_{\text{merged}} = \theta_{\text{base}} + \sum_{i=1}^{k} \lambda_i (\theta_i - \theta_{\text{base}})$$

LR Warmup + Cosine Decay:

$$\eta(t) = \begin{cases} \eta_{\max} \cdot \dfrac{t}{T_{\text{warmup}}} & t < T_{\text{warmup}} \\ \dfrac{\eta_{\max}}{2}\left(1 + \cos\left(\pi \cdot \dfrac{t - T_{\text{warmup}}}{T_{\text{total}} - T_{\text{warmup}}}\right)\right) & t \geq T_{\text{warmup}} \end{cases}$$

N-gram Overlap (Decontamination):

$$\text{overlap}(a, b) = \frac{|\text{ngrams}_n(a) \cap \text{ngrams}_n(b)|}{|\text{ngrams}_n(a)|}$$

Top Follow-up Q&As

1. **Q: Why is LoRA's B matrix initialized to zero while A is initialized with small random values?**
   - **A**: Initializing B to zero guarantees $\Delta W = BA = 0$ at the start of training regardless of A's values, giving a safe, deterministic starting point that exactly reproduces the base model's behavior before any updates.
2. **Q: Why does ZeRO-1 alone recover most of ZeRO's memory benefit?**
   - **A**: Because it shards the largest single term (12Ψ optimizer state) across GPUs first, before touching the smaller gradient and parameter terms.

3. **Q: Why is the reference model still required in DPO if there's no reward model?**
    - **A**: It anchors the implicit reward — without it, the loss would only depend on the policy's own log-probabilities, which it could trivially inflate without any grounding in what the SFT model considered plausible.
4. **Q: Why does GRPO need multiple rollouts per prompt while DPO needs none?**
    - **A**: GRPO computes its advantage baseline from a sampled group's own statistics, requiring live generation; DPO computes log-probabilities on fixed, pre-collected preference pairs with no generation step at all.
5. **Q: What specific problem does TIES-Merging's "elect sign" step solve?**
    - **A**: When two task vectors disagree on the sign of a parameter's needed change, naive averaging lets them partially cancel; TIES resolves the conflict by keeping only entries agreeing with the majority-magnitude sign.
6. **Q: Why does a checkpoint need optimizer state, not just model weights?**
    - **A**: Resuming without Adam's momentum/variance buffers effectively resets optimizer statistics mid-run, reintroducing early-training-style instability even though the weights themselves are correct.
7. **Q: How is reward hacking different from ordinary overfitting?**
    - **A**: Both share a "proxy improving, true signal worsening" shape, but overfitting is a data-memorization problem while reward hacking is specifically about exploiting a flawed reward *objective*, independent of memorization.
8. **Q: Why can indirect data contamination happen even without directly including benchmark text?**
    - **A**: Synthetic training data generated by a model that was itself exposed to benchmark content during its own training can reproduce benchmark-adjacent patterns, contaminating your dataset indirectly.